Intelligent Systems with Applications 22 (2024) 200372



Contents lists available at ScienceDirect

Intelligent Systems with Applications

journal homepage: www.journals.elsevier.com/intelligent-systems-with-applications





Wind turbine fault detection and identification using a two-tier machine learning framework

Zaid Allal [a], Hassan N. Noura [b], Flavien Vernier [a], Ola Salman [c], Khaled Chahine [d,*]

[a] Université Savoie Mont Blanc, LISTIC, France
[b] Univ. Franche-Comté (UFC), FEMTO-ST Institute, France
[c] DeepVu, USA
[d] College of Engineering and Technology, American University of the Middle East, Kuwait

ARTICLE INFO

Keywords:
Renewable energy
Proactive maintenance
SCADA
Data balancing
Explainability
Ensemble learning

ABSTRACT

A proactive approach is essential to optimize wind turbine maintenance and minimize downtime. By utilizing advanced data analysis techniques on the existing Supervisory Control and Data Acquisition (SCADA) system data, valuable insights can be gained into wind turbine performance without incurring high costs. This allows for early fault detection and predictive maintenance, ensuring that unscheduled or reactive maintenance is minimized and revenue loss is mitigated. In this study, data from a wind turbine SCADA system in the southeast of Ireland were collected, preprocessed, and analyzed using statistical and visualization techniques to uncover hidden patterns related to five fault types within the system. The paper introduces a conditional function designed to test two given scenarios. The first scenario employs a two-tier approach involving fault detection followed by fault identification. Initially, faulty samples are detected in the first tier and then passed to the second tier, which is trained to diagnose the specific fault type for each sample. In contrast, the second scenario involves a simpler solution referred to as naive, which treats fault types and normal cases together in the same dataset and trains a model to distinguish between normal samples and those related to specific fault types. Machine learning models, particularly robust classifiers, were tested in both scenarios. Thirteen classifiers were included, ranging from tree-based to traditional classifiers, neural networks, and ensemble learners. Additionally, an averaging feature importance technique was employed to select the most impactful features on the model decisions as a starting point. A comparison of the results reveals that the proposed two-tier approach is more accurate and less time-consuming, achieving 95% accuracy in separating faulty from normal samples and approximately 91% in diagnosing each fault type. Furthermore, ensemble learners, particularly bagging and stacking, demonstrated superior fault detection and identification performance. The performance of the classifiers was validated using t-SNE and explainable AI techniques, confirming that the impactful features align with the findings and that the proposed two-tier solution outperforms the naive solution. These results strongly indicate that the proposed solution is accurate, independent, and less complex compared to existing solutions.

1. Introduction

The rapid expansion of renewable energy sources, especially wind power, has driven the widespread adoption of wind turbine systems globally (Allal, Noura, Salman & Chahine, 2024). In 2021, wind power contributed 9% of total electricity generation in the United States, making it the second-largest contributor to electric-generating capacity additions after solar energy (Wind Energy Technologies Office). These wind turbine systems play a critical role in meeting the increasing energy demands while concurrently reducing carbon emissions and combating climate change, thereby contributing to the objectives of the Paris Agreement, which seeks to limit global temperature rise to 1.5 °C to mitigate the severe impacts of climate change (Segger, 2016). Ensuring reliable and efficient operation is essential for sustainable energy production, as wind turbines face various operational and environmental challenges that can lead to malfunctions and faults. Such issues result in costly downtime periods, reduced energy output levels, and increased maintenance expenses. According to data from the European Wind Energy Association reported in 2009, approximately 30% of overall wind power generation costs is attributed to operations and maintenance expenditures.

* Corresponding author.
E-mail address: khaled.chahine@aum.edu.kw (K. Chahine).

https://doi.org/10.1016/j.iswa.2024.200372
Received 3 January 2024; Received in revised form 4 April 2024; Accepted 15 April 2024
Available online 26 April 2024
2667-3053/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

Monitoring wind energy systems ensures consistent energy production and prevents downtime. Conventional wind turbine fault detection and condition monitoring methods have relied on rule-based systems and physical modeling (Miguelánez & Lane, 2010). While these methods have demonstrated some success in detecting and diagnosing common faults, they struggle to adapt to the dynamic and complex nature of wind turbine systems, resulting in limited accuracy and higher false alarm rates. Additionally, periodic maintenance and inspection strategies may incur additional costs, especially when no faults are present during inspection. Moreover, the challenging deployment environments, often harsh and offshore, make regular visualization and inspection difficult. Consequently, many wind turbine systems have sensors and data collection tools, enabling real-time performance monitoring at specific timestamps. These advancements facilitate the timely identification of potential issues, allowing for proactive maintenance and optimization of the system's overall performance.

Proactive maintenance is anchored by two core pillars: predictive maintenance and preventive maintenance. These principles utilize historical data to establish predefined maintenance schedules or predict potential failures proactively. Such methodologies significantly enhance maintenance practices by identifying issues early and scheduling maintenance during downtime. With AI-driven advancements, Machine Learning (ML) has become pivotal in research and practical applications, notably improving predictive maintenance strategies. The integration of systems like anomaly detectors (Nassif, Talib, Nasir, & Dakalbab, 2021), early warning systems (Moon, Kim, Lee, & Moon, 2019), and demand forecasting tools (Carbonneau, Laframboise, & Vahidov, 2008) has become more seamless and less error-prone, thanks to the capabilities provided by ML technologies. The widespread adoption of data-driven approaches is fueled by the increasing availability of data in recent years, coupled with the growing complexity of designing model-based fault detection systems as systems become more intricate. With technologies like the Internet of Things (IoT) and the digitization of various industries, vast amounts of data are being generated at an unprecedented rate. This wealth of data is a valuable resource for developing and training sophisticated ML models. Designing model-based fault detection systems can become increasingly challenging as systems become more complex. The complexity of modern systems, such as industrial processes or intricate machinery, often involves intricate relationships and dependencies between various components. Accurately modeling these systems can be daunting, particularly considering the potential interactions and nonlinearities. In contrast, data-driven approaches offer a more flexible and adaptable solution to fault detection. By leveraging available data, these approaches can capture underlying patterns and anomalies indicative of faults without requiring an in-depth understanding of the system's internal mechanisms. This makes data-driven techniques particularly well-suited for addressing faults in complex systems where traditional model-based approaches may struggle to keep pace with evolving dynamics. ML utilizes historical and real-time data to predict the future behavior of systems. By employing statistical algorithms, it uncovers hidden patterns in the data, enabling the creation of predictive models that enhance understanding, manipulation, and detection of anomalous behavior (Carvalho et al., 2019; Durocher & Feldmeier, 2004; Paolanti et al., 2018; Susto, Schirru, Pampuri, McLoone, & Beghi, 2014).

Fault detection is crucial in predictive and preventive maintenance techniques, enabling timely diagnosis and mitigation of potential equipment failures. It is a vital component of maintenance management, providing early warnings and insights into potential failures or deviations from normal operating conditions. Fault detection systems continuously monitor equipment and analyze data from various sensors and sources to detect anomalies, unusual patterns, or signs of degradation. Based on identified flaws, maintenance teams can take proactive measures, such as employing predictive maintenance procedures, to prevent minor issues from escalating into major breakdowns. Utilizing ML for such objectives, especially in energy systems where maintenance costs are significant and operating conditions are harsh, holds significant promise (Forootan, Larki, Zahedi, & Ahmadi, 2022; Liu et al., 2021; Wang, Poovendran, & Manokaran, 2021). Furthermore, fault detection complements preventive maintenance efforts by highlighting critical areas requiring frequent inspections or targeted maintenance interventions, thereby enhancing equipment reliability and durability. Existing fault detection and identification solutions, particularly in earlier works, can be categorized into two main aspects. Firstly, the accuracy of the obtained results often falls short of desired levels for various reasons. This includes instances where an inappropriate model is selected for the task, often resulting from initially simplistic choices that are later refined. Additionally, suboptimal results can be attributed to preprocessing steps integrated into the pipeline, extensively discussed in numerous papers (e.g., Alshdaifat, Alshdaifat, Alsarhan, Hussein, and El-Salhi (2021), Crone, Lessmann, and Stahlbock (2006), Uysal and Gunal (2014)). Secondly, addressing the complexity of the problem is crucial. Many solutions overlook the inherent complexity, leading to what can be termed a naive solution. Essentially, this approach fails to account for the nuanced nature of fault detection and identification. A more effective strategy involves breaking down the problem into two subproblems: first, detecting faulty instances without considering the specific fault type, and second, diagnosing the precise fault type for the identified faulty samples. This approach, often referred to as a two-tier system, significantly reduces the complexity from the number of faults plus one in a single system to binary fault detection and a variable number of faults for fault identification. To comprehensively address these challenges, emerging solutions leverage ensemble learning capabilities. By employing a two-tier system with diverse ensemble learning combinations, these solutions offer improved performance and dynamic solutions in fault detection and identification. Moreover, the flexibility of the ensemble learning approach allows for customization of the pipeline to accommodate varying preprocessing requirements, further enhancing the effectiveness of the overall system.

1.1. Motivation and contributions

The primary focus of this study revolves around the issues associated with wind turbines, emphasizing the need to address each concern individually. The overarching goal is to ensure the optimal utilization of these energy converters, leading to enhanced performance and reduced maintenance costs. The motivation driving this work is intrinsically aligned with key aspects that enhance renewable energy systems. Each aspect serves as a fundamental pillar shaping the research objectives and impact:

- **Environmental Sustainability**: Developing an effective fault detection system for wind turbines directly promotes sustainable energy production crucial for the environment. By ensuring optimal performance and minimizing downtime caused by unexpected failures, this system aids in maximizing the utilization of wind energy resources.
- **Economic Viability**: Unplanned maintenance and downtimes in wind turbines can lead to substantial financial losses. Implementing a robust fault detection system can reduce maintenance costs and extend the lifespan of wind turbine assets. This results in maximizing energy production and enhancing the economic viability of wind energy projects.
- **Grid Integration and Stable Energy**: Timely identification and resolution of faults by a fault detection system guarantee stable power supply and seamless integration into the grid. The system contributes to grid stability and reliable energy distribution by avoiding disruptions arising from undetected faults.
- **Optimized Maintenance**: Integrating ML into maintenance and fault detection leads to optimized asset management and decreased maintenance costs (Ogunfowora & Najjaran, 2023; Ran,

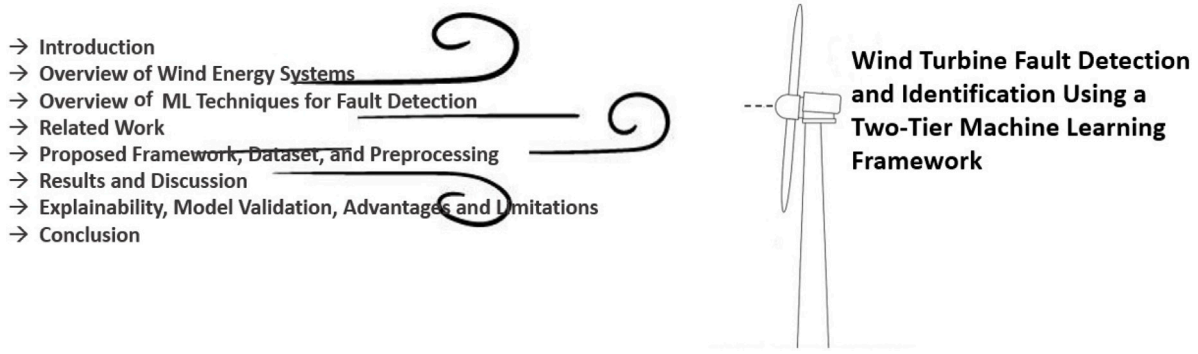


Fig. 1. The paper outline.

Zhou, Lin, Wen, & Deng, 2019). The "maintenance where needed, when needed" concept ensures resources are utilized efficiently and maintenance tasks are performed precisely when necessary.

In pursuit of these objectives, the paper aims to:

- **Conduct** a thorough examination of fault types associated with wind energy systems, specifically focusing on identifying and understanding the various types of faults that can occur in wind turbines. This analysis aims to provide valuable insights into the root causes of faults, their patterns, and their overall impact on system performance.
- **Develop** a two-tier framework specifically designed for detecting and diagnosing faults in wind energy converters. This framework strategically integrates ensemble learners first to address fault detection and subsequently diagnose each fault type.
- **Contribute** to the field by implementing over 13 classifiers from diverse categories. This comprehensive analysis is a valuable benchmark for future research endeavors in the same domain. By extensively comparing different classifiers, this work sets the stage for advancements and innovations in fault detection methodologies.
- **Validate** the effectiveness of the proposed approach through a succinct comparative analysis. The comparison underscores the superior accuracy achieved in both fault detection and identification.

The paper progresses as illustrated in Fig. 1. Beginning with an introduction, it provides an overview of wind energy systems, addressing topics such as wind turbine converters, system faults, and problem formulation. This is followed by a concise literature review that examines ML algorithms within this context, discussing ML models, classification metrics, and explainability tools. Subsequently, the previous works section is presented. The proposed framework is then elaborated upon, followed by the results and discussion section. Here, findings from the naive system are discussed, along with results from the two-tier framework, including optimization details and a comparison with the naive solution. An exploration of the models' explainability and interpretability, along with their respective advantages and limitations, follows. Finally, the paper concludes as it reaches its endpoint.

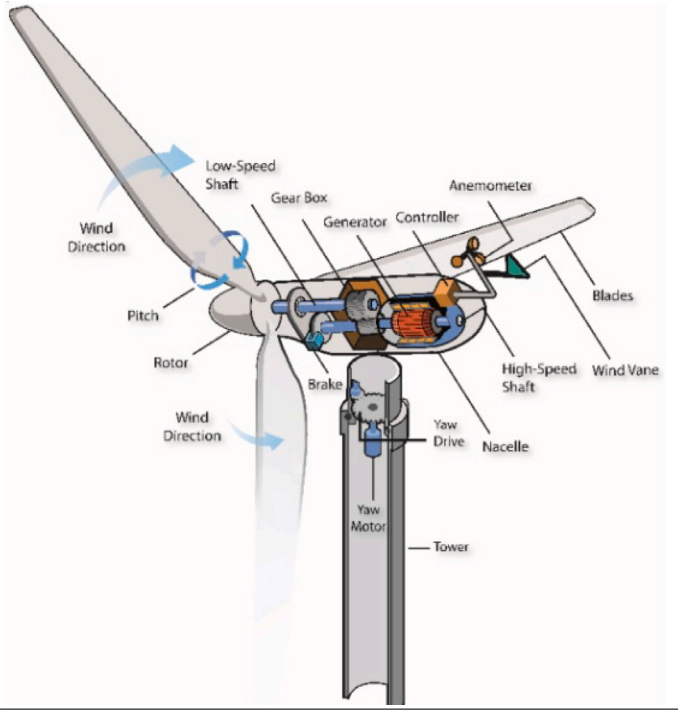
2. Overview of wind energy systems

Wind energy systems are technological solutions designed to harness the kinetic energy of wind currents and convert it into either mechanical or electrical energy. This converted energy serves various purposes, with electricity generation being the primary application. Wind energy represents a renewable energy source that capitalizes on the inherent potential of wind dynamics to produce economically sustainable and environmentally friendly power (Saidur, Rahim, Islam, & Solangi, 2011). A wind electric system typically consists of multiple wind turbines, often configured as horizontal three-bladed units mounted on elevated towers to effectively capture wind streams (US Department of Energy, 2023). The system relies on the interaction of various components, including blades, low-speed and high-speed shafts, a gearbox, a generator, a yaw mechanism, a tower, a brake system, and a controller. The process begins with the kinetic energy of the wind interacting with the turbine's blades. These aerodynamically designed blades capture energy from the moving air, initiating their rotation when wind speeds reach or exceed the turbine's cut-in speed. As the blades rotate, they transfer rotational force to a low-speed shaft. This rotational movement is then transmitted to a gearbox, which amplifies the rotation speed, converting the slow rotation of the blades into a higher speed suitable for electricity generation. The increased rotational speed from the gearbox is directed towards the generator, which utilizes electromagnetic principles to convert mechanical energy from the spinning shafts into electrical energy in the form of alternating current (AC) electricity (Tawfiq, Mansour, Ramadan, Becherif, & El-kholy, 2019). Wind turbines are equipped with a yaw mechanism to optimize energy capture efficiency by allowing them to rotate around a vertical axis, ensuring that the blades are constantly aligned with the wind direction, even as it changes. This ability to orient towards the wind maximizes energy extraction. The tower serves a dual purpose, providing elevation to position the blades where wind currents are stronger and as a sturdy support for the entire turbine system. A brake system is incorporated to control blade rotation during maintenance or when wind speeds are too high for safe operation. The controller, a sophisticated electronic component, monitors wind speed, direction, generator output, and system status, adjusting parameters such as blade pitch and yaw orientation to optimize turbine efficiency and ensure safe operation. The harmonious interconnection of these components orchestrates the transformation of wind kinetic energy into valuable electrical power output based on the following governing equations (Kumar, Khan, & Pandey, 2018). First, the kinetic energy is defined by $Ke = \frac{1}{2}mv^2$, which, when multiplied by the time unit, gives the power. The mass flow rate is given by fluid mechanics as $\frac{dm}{dt} = \rho A v$ where A is the rotor swept area, ρ is the air density, and v is the velocity. This provides the final equation of wind power $P = \frac{1}{2}\rho A v^3$. Table 1 provides the critical components of such a system and a representation of all parts of a wind turbine (Fu, Gao, Liu, Zhang, & Yin, 2020).

2.1. Wind turbine types

There are two primary types of wind turbines currently in use: gearbox wind turbines and direct-drive wind turbines, as illustrated in Fig. 2. Gearbox turbines incorporate a gearbox between the low-speed rotor and a higher-speed electrical generator. In contrast, direct-drive turbines lack a gearbox and instead employ a synchronous generator directly powered by the rotor. The gearbox is the component requiring the highest maintenance within the turbine. Eliminating it could significantly enhance reliability. However, direct-drive generators still

Table 1
The components and representation of a wind turbine (Fu et al., 2020).

Wind Turbine Components	Representation
• **Rotor Blades**: aerodynamically designed blades that capture the wind's energy. The force of the wind causes the blades to rotate. • **Hub**: connects the rotor blades to the central shaft. • **Nacelle**: the housing that contains the gearbox, generator, and other critical components. It is located atop the tower. • **Tower**: a structural foundation for the wind turbine. It supports the turbine and elevates it to a sufficient height, thereby facilitating the harnessing of substantial kinetic energy from powerful wind currents.	

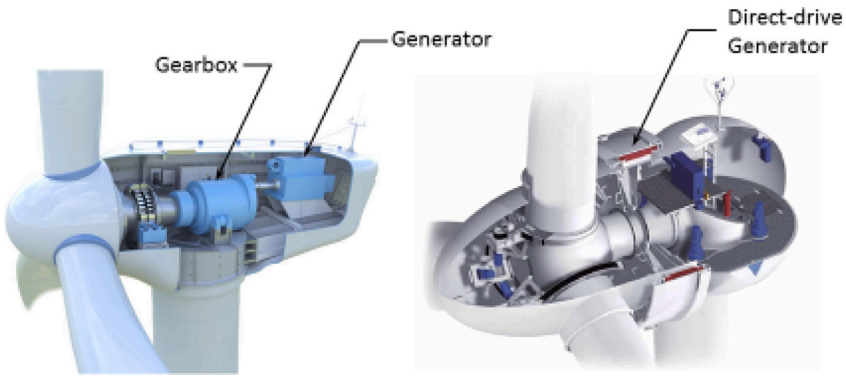


Fig. 2. Wind turbine categories: gearbox versus direct drive (Advanced Wind Turbine Drivetrain Trends and Opportunities Department of Energy, 2019).

face challenges regarding cost and weight compared to gearbox models, which have been prevalent until now. As of the current date, neither technology has gained clear dominance, resulting in ongoing competition between the two.

2.2. Fault types in wind energy systems

Numerous factors influence wind conditions, blade design, rotor dynamics, bearing integrity, generator functionality, gearbox performance, pitch control, and more. These factors can cause various faults, encompassing blade-related problems, generator failures, gearbox anomalies, pitch mechanism irregularities, and yaw system deviations (Zhang et al., 2020). Based on the related equipment, we can classify faults in wind turbines as presented in Table 2.

2.3. Problem formulation

The fault detection problem can be formulated as follows. Let $X = \{x_1, x_2, \ldots, x_n\}$ be the set of input data samples, where each sample $x_i = [x_{i1}, x_{i2}, x_{i3}, \ldots .x_{im}]$ represents a vector of m features obtained from the wind turbine sensors and operational parameters. The output of a fault detection system can be formulated naively by directly considering faulty types and non-faulty samples in the same system. In this case, the equation becomes:

$$\hat{Y} = \begin{cases} N_f, \text{ non-faulty} \\ f_1, \text{ fault type 1} \\ f_2, \text{ fault type 2} \\ \vdots \\ f_n, \text{ fault type n} \end{cases}$$

Alternatively, the system can be a two-tier approach. First, it detects the fault/normal cases, and then it targets the faulty sample identification using a convenient approach, which results in identifying the fault type in the considered sample. The equations of the two-tier approach are as follows:

Table 2
Fault types in wind turbines.

Blade-Related Faults: 1. Blade Misalignment 2. Blade Erosion and Wear	Generator Faults: 1. Electrical Winding Failures 2. Power Output Irregularities
Gearbox Anomalies: 1. Gear Tooth Wear 2. Bearing Failures	Pitch Mechanism Irregularities: 1. Pitch Actuator Malfunctions 2. Blade Angle Inconsistencies
Other Faults: 1. Icing and Cold Weather 2. Lightning Strikes & Harsh Wind Conditions	Yaw System Deviations: 1. Yaw Drive Failures 2. Yaw Control Inaccuracies

- Fault Detection: The fault detection system generates an output $\hat{Y}_{Dt}$ expressed as:

$$\hat{Y}_{Dt} = \begin{cases} 1, & \text{if } x_i \text{ is faulty} \\ 0, & \text{if } x_i \text{ is non-faulty} \end{cases}$$

- Fault Identification: when the fault detection output is equal to 1, the fault identification system generates an output $\hat{Y}_{Dg}$ expressed as:

$$\hat{Y}_{Dg} = \begin{cases} f_1, \text{ fault type 1} \\ f_2, \text{ fault type 2} \\ \vdots \\ f_n, \text{ fault type n} \end{cases}$$

where $F = \{f_1, f_2, \ldots, f_n\}$ are the fault types, $\hat{Y}$ is the output of the ML model, and n is the number of faults in the dataset.

The main objective of the fault detection system is to acquire a mapping function $f : X \rightarrow \hat{Y}$ capable of precisely categorizing the input data samples into one of two scenarios. Firstly, the samples should be accurately classified as either faulty or non-faulty. Secondly, it should directly identify the specific fault type if a fault is present. This mapping function will be constructed by first training ML models on historical data in an offline system and then exposing the ML model to online data from the wind turbine SCADA system. The fault detection and identification system can be coupled with predictive models that forecast the next feature values with a proactive approach. The research will explore and evaluate different ML algorithms and propose an ensemble approach combining the strengths of multiple techniques to enhance fault detection performance. The performance of the fault detection system will be assessed using appropriate evaluation metrics, such as accuracy, precision, recall, F1-score, and area under the ROC curve, to quantify the effectiveness of the proposed method in detecting and classifying faults in wind turbines. It is important to emphasize that the metrics discussed are widely recognized in the literature when evaluating classification problems. They offer a valuable means of measuring the model's ability to make correct predictions and highlighting areas where improvements or adjustments may be necessary to enhance classification outcomes.

3. Overview of ML techniques for fault detection

ML, a subset of artificial intelligence, encompasses a wide range of algorithms tailored for various tasks. These tasks include regression, which predicts continuous outputs, and classification, where the goal is to assign the correct class to a given sample in either a binary or multiclass system. Additionally, ML extends to tasks such as clustering and involves sophisticated Deep Learning (DL) models for solving complex problems. When dealing with fault detection and identification, the challenge becomes a classification problem. In this context, multiple classifiers play a crucial role, offering a range of options from tree-based models and neural networks to logistic regression and beyond. The scikit-learn library encompasses several of these algorithms, providing a convenient platform for implementation (Pedregosa et al., 2011). However, certain algorithms, due to their intricacies and specific requirements, may necessitate dedicated platforms beyond scikit-learn. Exploring the literature reveals a variety of algorithms, each with its strengths and applications. This study harnesses and explores specific algorithms tailored to fault detection and identification complexities. Among these, tree-based models, neural networks, and logistic regression are utilized, each offering unique advantages in addressing the challenges inherent in classifying faults. Ultimately, the selection of algorithms is a critical consideration, influenced by the nature of the problem, the available data, and the desired outcomes. The subsequent sections will delve into a detailed exploration of the algorithms found in the literature and those specifically employed in the current study, highlighting their merits and contributions to the field of fault detection and identification.

3.1. Machine learning algorithms

In this study, we pursued the objective of fault detection by training a diverse range of ML models. We thoroughly explored solo and ensemble learners to evaluate their performance comprehensively.

- **XGBoost (Extreme Gradient Boosting)**: XGBoost is a powerful and efficient implementation of gradient-boosting machines. It is widely used for supervised learning tasks, particularly in structured/tabular data settings. XGBoost sequentially builds an ensemble of weak decision trees, with each subsequent tree aiming to correct the errors made by the previous ones. It incorporates regularization techniques to prevent overfitting and utilizes advanced optimization strategies to enhance computational efficiency (Chen & Guestrin, 2024).
- **CatBoost**: CatBoost is a gradient-boosting library developed by Yandex that is designed to handle categorical features seamlessly. It employs an innovative algorithm for gradient boosting, which automatically handles categorical variables without manual encoding. CatBoost also incorporates various optimizations to improve training speed and predictive performance, making it particularly well-suited for datasets with a mix of categorical and numerical features (Dorogush, Gulin, & Matveyev, 2024).

- **Extra Trees (Extremely Randomized Trees)**: Extra Trees is an ensemble learning method that builds multiple decision trees and combines their predictions through averaging or voting. What sets Extra Trees apart from traditional random forests is that it randomly selects feature subsets at each node and makes split decisions based on random thresholds. This randomness introduces additional diversity among the trees, sometimes leading to better generalization performance, especially in high-dimensional datasets (Alfian et al., 2022).
- **Random Forest**: Random Forest is a popular ensemble learning technique based on decision trees. It constructs a multitude of decision trees during training and outputs the class that is the mode of the classes (classification) or the mean prediction (regression) of the individual trees. Random forests introduce randomness in selecting the data points used for training each tree and the features considered for splitting at each node. This randomness helps reduce overfitting and improve the model's robustness (Allal, Noura & Chahine, 2024).
- **Multi-Layer Perceptron (MLP)**: MLP is a type of artificial neural network (ANN) characterized by having multiple layers of nodes, each connected to the next in a feedforward manner. It consists of an input layer, one or more hidden layers, and an output layer. MLPs can learn complex nonlinear relationships in data and are widely used for tasks such as classification and regression. Training an MLP involves forward propagation of inputs through the network, followed by backpropagation of errors to adjust the connection weights using gradient descent or its variants (Taud & Mas, 2018).
- **Ensemble Learning Models**: Ensemble learning models combine multiple individual models (base learners) to improve predictive performance (Dong, Yu, Cao, Shi, & Ma, 2020; Sagi & Rokach, 2018). There are several types of ensemble methods, including:
  - Boosting: Builds an ensemble of weak learners sequentially, where each subsequent learner focuses on the examples that previous models have misclassified. Popular boosting algorithms include AdaBoost, Gradient Boosting Machines (GBM), XGBoost, LightGBM, and CatBoost.
  - Bagging (Bootstrap Aggregating): Constructs multiple base learners in parallel, each trained on a bootstrap sample of the training data. The final prediction is typically the average (for regression) or majority vote (for classification) of the predictions from individual models.
  - Voting: Combines predictions from multiple base learners using a simple majority vote (for classification) or averaging (for regression). This method works well when the base learners have diverse strengths and weaknesses.
  - Stacking: Combines predictions from multiple base learners using a meta-learner, often a simple linear model. The base learners' predictions serve as input features to the meta-learner, which learns to combine them optimally to produce the final prediction.

Fig. 3 provides the principle of calculating the target output for the ensemble learning models, and Table 3 presents the target equations for the employed ML classifiers (solo learners). These classifiers were meticulously evaluated to ascertain their suitability for fault detection within wind energy systems. Through this rigorous exploration, we aim to contribute valuable insights into the optimal utilization of ML techniques to improve fault detection reliability in renewable energy infrastructure.

3.2. Classification metrics

Classification metrics serve as a vital tool for evaluating the performance of ML models. They furnish a numerical structure for evaluating prediction quality, facilitating informed choices, and model improvements. They aim to assess a classifier's efficacy by gauging accuracy, precision, recall, and other metrics. Table 4 provides a brief description of the classification metrics used in this work, where T_{ii} are the instances belonging to class i and truly classified as i instances, and T_{ij} are the instances belonging to class i and falsely classified as j instances. In this study, we employ several key metrics, including accuracy, precision, recall, and F1-score, alongside the representation of a confusion matrix. These metrics are widely utilized in classification problems due to their ability to offer valuable insights into the overall performance of the model and the accuracy of predictions for each class. Accuracy is a fundamental measure, expressing the overall correctness of the model's predictions. Precision, on the other hand, gauges the accuracy of positive predictions, emphasizing the ratio of true positives to the total predicted positives. Recall, also known as sensitivity or true positive rate, assesses the model's ability to correctly identify all relevant instances within a given class. The F1-score strikes a balance between precision and recall, providing a harmonized evaluation metric. Including a confusion matrix further enhances our understanding of the model's performance. This matrix delineates the distribution of true positive, true negative, false positive, and false negative predictions for each class, facilitating a comprehensive analysis of classification outcomes.

3.3. Explainability and interpretability

Explainability and interpretability are crucial aspects in ML applications, especially in domains such as energy, healthcare, and economy, where transparency and understanding of model decisions are paramount. Explainability refers to the capacity of a model to provide clear and understandable reasons for its predictions or decisions. On the other hand, interpretability is broader, encompassing the ease with which humans can comprehend the overall functioning of a model, including its features, parameters, and decision-making processes. These two aspects are presented in the following:

- **eXplainable AI:** The ability of AI systems to provide understandable explanations for their decisions and actions is referred to as XAI or Explainable Artificial Intelligence. It tries to improve the transparency and interpretability of AI models and their consequences for humans. This is especially true in sectors where AI judgments have substantial real-world implications, such as healthcare, banking, and autonomous cars (Gunning et al., 2019). Some of its best-known tools are presented in Fig. 4.
- **t-SNE:** visualizes high-dimensional data by giving each data point a location in a two or three-dimensional map. The technique is a variation of the Stochastic Neighbor Embedding (Hinton & Roweis, 2002) that is much easier to optimize and produces significantly better visualizations by reducing the tendency to crowd points together in the center of the map. t-SNE is better than existing techniques at creating a single map that reveals structure at many scales. This is particularly important for high-dimensional data on several different but related low-dimensional manifolds, such as images of objects from multiple classes seen from multiple viewpoints (Van der Maaten & Hinton, 2008).

4. Related work

When examining the literature regarding fault detection and identification in wind energy converter systems, one encounters a range of approaches. These approaches vary from simplistic methods, which incorporate a comprehensive dataset containing samples of both fault types and normal operations into a single prediction process, to more sophisticated strategies. In the former, evaluating the wind turbine's condition involves classifying the data into the number of fault types present plus a class representing normal operation. Conversely, alternative methodologies adopt a two-step approach. All fault types are

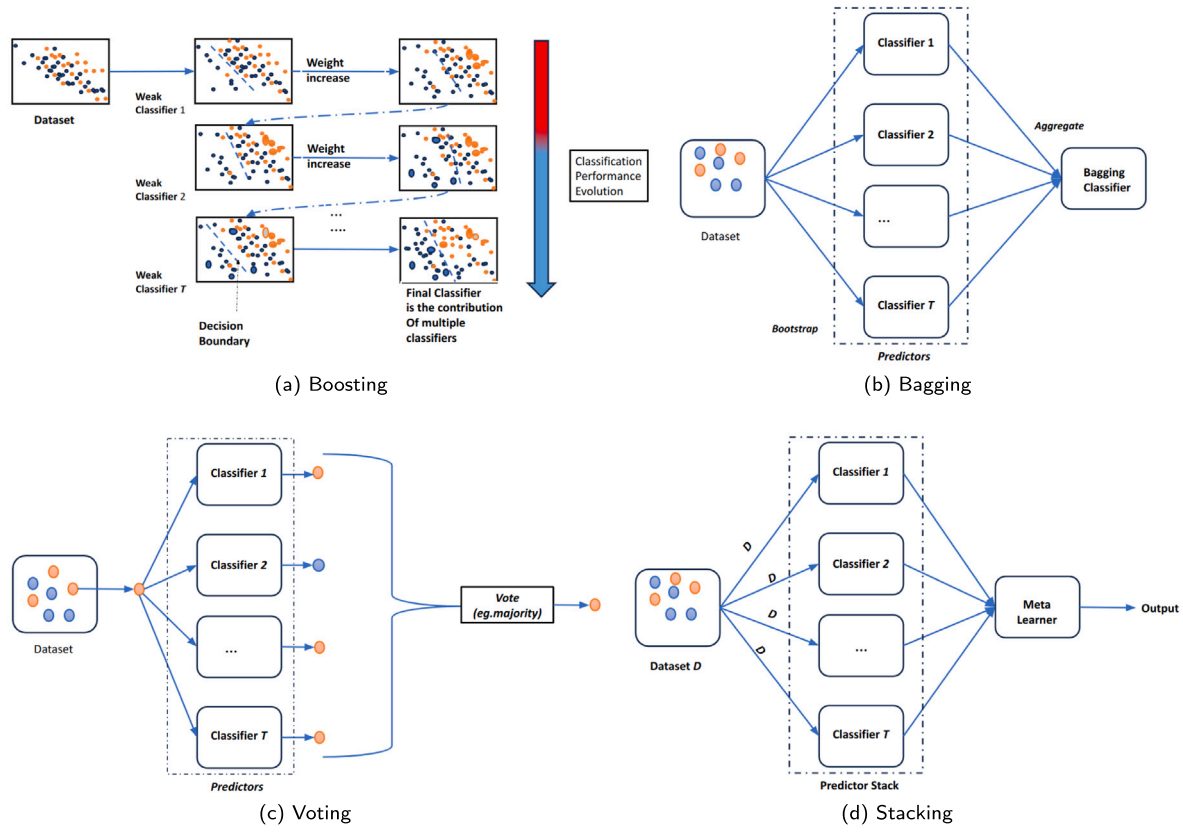


Fig. 3. Ensemble machine learners.

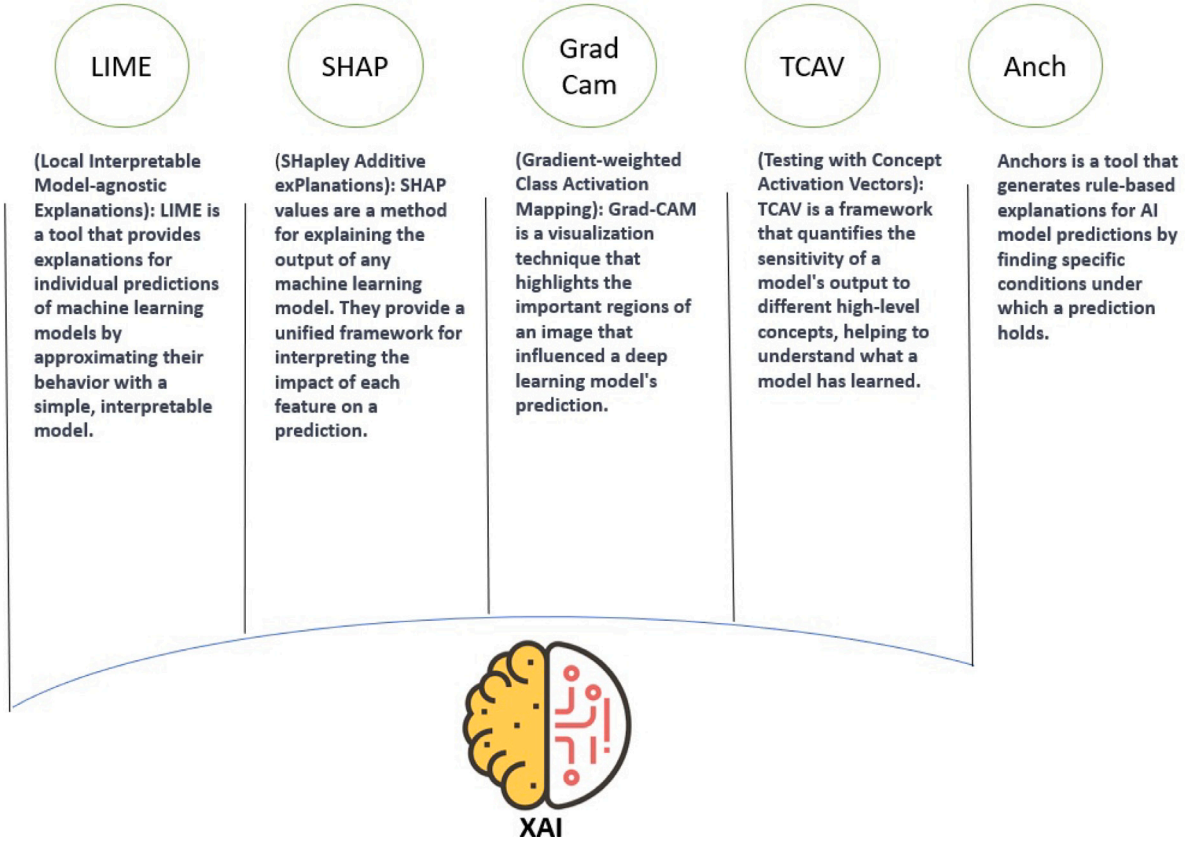


Fig. 4. XAI tools.

Table 3
ML models and equations.

Model	Target equation	Equation parameters
Extra Trees (Geurts, Ernst, & Wehenkel, 2006)	$\hat{y}_{\text{Extra Trees}} = \frac{1}{N}\sum_{i=1}^{N} T_i(x)$	• $\hat{y}_{\text{Extra Trees}}$: Predicted output. • N: Number of decision trees in the ensemble. • $T_i(x)$: Prediction of the ith decision tree.
Random Forest (Cutler, Cutler, & Stevens, 2012)	$\hat{y}_{\text{Random Forest}} = \frac{1}{N}\sum_{i=1}^{N} T_i(x)$	• $\hat{y}_{\text{Random Forest}}$: Predicted output. • N: Number of decision trees in the ensemble. • $T_i(x)$: Prediction of the ith decision tree.
MLP (Taud & Mas, 2018)	$\hat{y}_{\text{MLP}} = f(W^{(2)} \cdot f(W^{(1)} \cdot x + b^{(1)}) + b^{(2)})$	• $\hat{y}_{\text{MLP}}$: Predicted output. • $W^{(1)}$ and $b^{(1)}$: Weights and biases of the first hidden layer. • $W^{(2)}$ and $b^{(2)}$: Weights and biases of the output layer.
XGBoost (Chen et al., 2015)	$\hat{y}_{\text{XGBoost}} = \sum_{i=1}^{N} f_i(x)$	• $\hat{y}_{\text{XGBoost}}$: Predicted output. • N: Number of weak learners. • $f_i(x)$: Prediction of the ith weak learner.
Histogram Gradient Boosting (Xia, Zheng, Li, Huang, & Gao, 2023)	$\hat{y}_{\text{Histogram Gradient Boosting}} = \sum_{i=1}^{N} f_i(x)$	• $\hat{y}_{\text{Histogram Gradient Boosting}}$: Predicted output. • N: Number of weak learners. • $f_i(x)$: Prediction of the ith weak learner.
CatBoost (Jabeur, Gharib, Mefteh-Wali, & Arfi, 2021)	$\hat{y}_{\text{CatBoost}} = \sum_{i=1}^{N} f_i(x)$	• $\hat{y}_{\text{CatBoost}}$: Predicted output. • N: Number of weak learners. • $f_i(x)$: Prediction of the ith weak learner.

Table 4
Classification evaluation metrics.

Metric type	Metric name and description	Formula	Optimal value
Binary Classification	**False Positive (FP)**: Incorrectly classified as positive when actually negative.	FP	0
	False Negative (FN): Incorrectly classified as negative when actually positive.	FN	0
	Accuracy: Measures the overall correctness of the classification.	$\text{Acc}=\frac{TP+TN}{TP+TN+FP+FN}$	1
	Precision: Ratio of True Positives to the total number of positive predictions.	$\frac{TP}{TP+FP}$	1
	Recall (Sensitivity): Ratio of True Positives to the total number of actual positives.	$\frac{TP}{TP+FN}$	1
	F1-score: Harmonic mean of precision and recall.	$2 \cdot \frac{\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$	1
	AUC (Area Under the ROC Curve): Metric used to evaluate the performance of a binary classification model.	–	1
Multiclass Classification	**False Positive (FP)**: Misclassified instances for each class.	FP	0
	False Negative (FN): Misclassified instances for each class.	FN	0
	Accuracy: Ratio of correctly classified instances for all classes to the total number of instances.	$\frac{\sum_{i=1}^{n} T_{ii}}{\sum_{i=1}^{n}\sum_{j=1}^{n} T_{ij}}$	1
	Precision (Per Class): Precision calculated separately for each class.	$\frac{T_{ii}}{\sum_{j=1}^{n} T_{ji}}$	1
	Recall (Per Class): Recall calculated separately for each class.	$\frac{T_{ii}}{\sum_{j=1}^{n} T_{ij}}$	1
	F1-score (Per Class): F1-score calculated separately for each class.	$2 \cdot \frac{\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$	1

initially treated as a single class alongside the normal operating state, thus framing the issue as a binary classification task. Subsequently, after successfully detecting faulty samples, a separate classifier addresses the identification of each specific fault type among the designated fault classes. In Chen, Xu, Zhang, and Zhang (2019), an innovative methodology based on DL neural networks was introduced for detecting faults in wind turbines. The primary focus of this method was to effectively address the challenges posed by imbalanced datasets, particularly cases where anomalous and normal data instances are unevenly distributed within fault turbine datasets. The study reported compelling results compared to conventional normal behavior methods when applied to a SCADA data system encompassing two turbines that exhibited blade icing accretion faults. In Leahy et al. (2018), the authors utilized a wind turbine SCADA dataset containing fault alarm data from the southern coast of Ireland. Their objective was to construct a fault detection classification system based on Support Vector Machines (SVMs), which involved training the SVMs with re-sampled data and using coupled ensemble SVMs. The results demonstrated the proposed approach's effectiveness, with recall scores exceeding 80%. Additionally, the developed system could predict possible faults up to 12 h in advance, providing valuable information for preventive maintenance and operational efficiency. Researchers in Leahy, Hu, Konstantakopoulos, Spanos, and Agogino (2016) employed the same wind turbine SCADA dataset and adopted a classification approach involving the initial analysis and data filtering from the SCADA system to target fault and fault-free conditions. The goal was to develop

advanced diagnosis and prediction systems tailored to specific faults. The researchers trained an SVM model for this purpose, achieving a poor precision of 8% and a recall of 90% for distinguishing between faulty and non-faulty cases. In De Sousa, Navar de Medeiros, Almeida, Reboucas Filho, de Albuquerque, et al. (2019), an intelligent system was proposed to detect incipient faults in the electric generators of wind turbines. The approach employed four different feature extraction methods on vibrant signals and tested three classifiers—Naive Bayes, SVM, and MLP. The system achieved high precision, correctly identifying 94.44% of normal conditions and showing low false positive and negative rates of 0.4%. In Xiang, Wang, Yang, Hu, and Su (2021), the authors utilized Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) DL models for detecting faults in wind turbine systems using SCADA data. An embedded attention mechanism was experimented with these models to enhance the significance of important features. The proposed system outperformed other approaches regarding a comprehensive evaluation index and effectively minimized false positive detection. In Helbing and Ritter (2018) presented a comprehensive review of DL frameworks applied to condition monitoring in wind turbines, addressing the challenge of unbalanced data in fault detection. Using DL techniques, the authors aimed to improve the accuracy and efficiency of fault detection in wind turbines. In Sun et al. (2021), a comprehensive review of machine-learning fault detection methods was presented, covering traditional ML, DL, ANNs, and transfer learning algorithms. The paper thoroughly analyzed and discussed the advantages and disadvantages of each system and offered information on future research directions and potential areas for further exploration. It emphasized the ability of diagnosis approaches to effectively recognize fault patterns, highlighting their potential for real-world applications. However, it also noted that DL-based diagnosis methods require sufficient labeled data to achieve optimal performance, presenting a challenge in scenarios where labeled data are limited. In Cheng et al. (2022), the challenges encountered in wind turbine systems, such as time delays and data dropouts, were addressed by developing a control strategy ensuring finite-time stability and dissipativity performance. Employing a hidden Markov model, the paper designed a finite-time asynchronous output feedback controller to estimate the current system modes. The controller achieved finite-time boundedness of system states and ensured strict dissipativity performance, effectively adapting to asynchronous situations. In Stojanović (2023), the challenge of maintaining control and stability in hydraulic servo actuators amid unknown system dynamics, unmeasurable states, and actuator faults was tackled. Utilizing adaptive dynamic programming (ADP), the authors employed a data-driven approach to iteratively compute an optimal control law. The proposed strategy achieved asymptotic tracking of desired trajectories despite actuator faults by integrating fault compensation techniques within the ADP framework. Theoretical analysis confirmed the convergence of this control approach, showcasing its ability to handle uncertainties and faults effectively. In Song, Sun, Song, and Stojanovic (2023), the authors presented a novel control strategy to address the complexities of large-scale systems with fractional-order dynamics in the presence of uncertainties and faults. The proposed scheme, combining dynamic surface control (DSC) with adaptive neural networks, guaranteed finite-time convergence and resilience against sensor-actuator faults. By adapting to complex dynamics using fractional-order models and neural networks, this control strategy offered a robust solution for maintaining stability and performance in challenging environments. In Wang, Zhuang, Tao, Paszke, and Stojanovic (2023), the authors proposed a two-pronged approach to ensure reliable control for Multi-Input Multi-Output (MIMO) systems susceptible to actuator faults and repetitive tasks. Combining Q-learning for fault estimation with norm-optimal iterative learning control (NOILC) for fault tolerance, this approach continuously monitored and adapted to unknown and time-varying actuator faults, providing real-time estimates of their severity and impact. The NOILC algorithm adjusted the control signal based on these estimates, effectively counteracting the negative influence of faults and maintaining the desired system performance. Simulation results demonstrated the robustness and adaptability of this approach in handling actuator faults during repetitive tasks, offering promise for various applications requiring accurate control of MIMO systems under challenging conditions. In Kumar et al. (2024), a ML model was proposed to monitor the performance of a Kaplan turbine in a hydropower plant across different operating conditions. The objective was to detect anomalies and optimize performance for efficient energy generation. Historical data from a Kaplan turbine-based hydropower plant was utilized to develop the model. ANN demonstrated superior performance in predicting the machine's efficiency, with an R-squared value of 0.99966. In Hu et al. (2016), domain knowledge was used to eliminate irrelevant features and improve detection results. Using the new feature set improved the F1-score by 27%.

In Table 5, a comparison between the current and previous works is made. The multiple conclusions drawn from the review provide valuable guidance for researchers and practitioners in the fault detection domain, helping them understand the strengths and limitations of various machine-learning techniques and identify areas that warrant further exploration and improvement.

5. Proposed framework, dataset, and preprocessing

In the field of fault detection, the existing literature addresses the issue primarily through a straightforward approach known as the naive process. This approach involves training classifiers on a dataset to predict whether a given sample is normal or exhibits a specific type of fault. In this system, the classifier's decision boundaries must correspond to the established classes within the dataset, such as Non-Faulty, Fault_1, Fault_2, …, Fault_n. The system's task is to determine the case using pertinent features in a multiclass classification setup. While this direct approach seems intuitive, it can face challenges when dealing with unbalanced data sets, and the experimental results show that the reduction of data complexity obtained by using data imbalance treatments (DIT) techniques occurs mainly for highly imbalanced datasets. They also show that, for most of the DIT investigated, there is a correlation between the reduction in data complexity and the gain in predictive performance (Barella, Garcia, de Souto, Lorena, & de Carvalho, 2021). An imbalance occurs when certain classes possess a significantly larger number of instances than others. This imbalance can potentially undermine the model's performance, leading to prediction bias, overfitting results, misclassification of minority classes, and, consequently, erroneous outcomes. This naive methodology was evaluated within the scope of this study to gain deeper insight into its potential results.

5.1. Framework

The proposed framework consists of two tiers aimed at effectively processing and analyzing data. Initially, data preprocessing is conducted in the preliminary tier. Data acquired from the SCADA system is categorized into three tabular data types: SCADA monitoring data, turbine data, and fault data. These datasets are then filtered, merged, and indexed appropriately based on time. Subsequently, preprocessing steps involve identifying and interpolating missing values (NaN), as well as converting categorical features into numerical representations, resulting in a clean dataset. Following this, exploratory data analysis ensues, incorporating visualization techniques, outlier detection, correlation analysis between target and features, and applying a balancing process to ensure equitable distributions and fault-type representation percentages. A crucial role is implemented to streamline the two-tier situation. Initially, it merges all categories of faults into a unified group termed "fault" and divides samples into two datasets: one merging faulty and normal samples, constituting the binary fault detection

Table 5
Comparison of previous solutions: techniques, characteristics, and limitations.

Paper	Trained classifiers	Hybrid classification or Naive	Handles imbalanced data	Limitations
Leahy et al. (2018)	SVM	Hybrid (2 tiers: detection then identification)	Yes, with 4 techniques: Undersampling, Oversampling, General Ensembles, Class Weight	• Data shuffling may alter the temporal order of samples. • Limited range of tested models.
Leahy et al. (2016)	SVM	Naive and Hybrid	No	• No correlation analysis. • Data shuffled. • 38 features eliminated based on the author's domain knowledge. • Averaging sensors readings (sensors with same reading). • Individual scaling of features.
De Sousa et al. (2019)	MLP, OPF, SVM	Naive	No	• Different datasets used. • F1-score results up to 94%. • Handled only electrical faults.
Xiang et al. (2021)	CNN, LSTM, Attention Mechanism	Naive	No	Handled as a regression task.
Hu et al. (2016)	SVM	Naive	Yes	• Used domain knowledge to eliminate features. • Presented results are better. • Preprocessing involved removing potentially useful samples.
Proposed	**Wide range of classifiers:** • MLP Classifier • Decision Tree, Random Forest, Extra Trees • Gradient/Ada/HistGradient Boosting • SVM • XGBoost, CatBoost, LightGBM • Voting, Bagging, Stacking	**Naive and Two-tier**	**Yes (SMOTE)**	• Not 100% accurate. • Only one balancing strategy was tested. • More classifiers could be tested.

dataset, while the other exclusively comprises faulty samples, constituting the fault identification dataset. The first dataset is utilized for training and testing the first-tier ML models, which focus on predicting binary classes—identifying fault presence or normalcy. Detected faulty samples are subsequently forwarded to the second-tier ML models trained on the fault identification dataset. Here, the conditional function intervenes again, realigning faulty samples and assigning each to its respective fault class. The second tier is then trained to diagnose specific fault types present in the samples. The framework iterates through a loop of training, postprocessing, performance evaluation, and hyperparameter tuning to optimize results. Fig. 5 illustrates this framework, while Fig. 6 elucidates the disparity between the two-tier approach and the naive approach.

5.2. Dataset and exploratory data analysis

The dataset utilized in this study originates from a 3-MW direct-drive turbine (Nuwara, 2023). This turbine supplies power to a significant manufacturing facility specializing in biomedical devices in the southern region of Ireland. The data are extracted from the turbine SCADA system and categorized into two sets: "operational" data and "status" data. The timeframe covered by the data spans 11 months, from May 2014 to April 2015. Additionally, the dataset includes fault data stored in separate files. These files can be integrated by matching their timestamps. The timestamps play a crucial role in establishing an index that distinguishes between samples exhibiting faults and those operating normally. This index also aids in identifying each sample's specific fault type. The documented faults in the dataset are described in Table 6. The SCADA dataset consists of 66 columns, capturing sensor readings for factors such as wind speed, blade rotation status, operating hours, component temperatures, and more. The status data includes 8 columns corresponding to entities such as main status, hidden or sub-status, and a textual representation of the status. The fault dataset focuses specifically on fault types and detection times. Importantly, all these datasets are time-indexed, facilitating the merging process and enabling accurate fault observation at the appropriate time. Fig. 7 illustrates the temporal extension of the three datasets throughout the index under consideration. Table 7 lists and describes the collected features in the three datasets (SCADA, status, and fault).

5.3. Data wrangling and preparation

Preprocessing involved merging the data from the three datasets into a single data frame using a chronological index. To accommodate the varying sizes of the datasets, the merging script meticulously handled missing data. Specifically, all samples lacking fault information

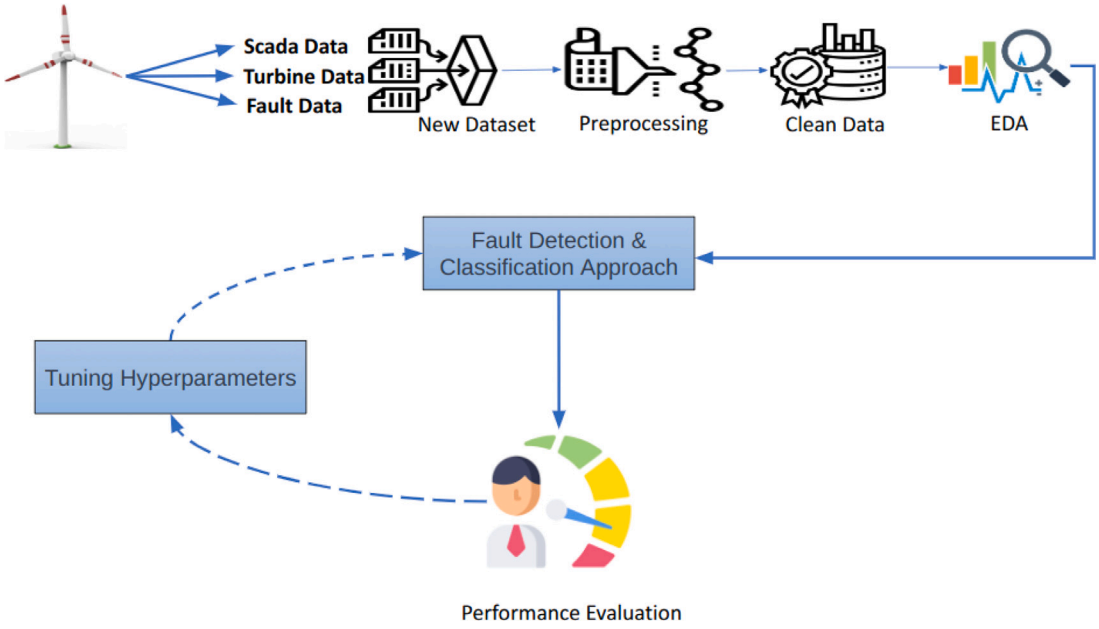


Fig. 5. The proposed framework.

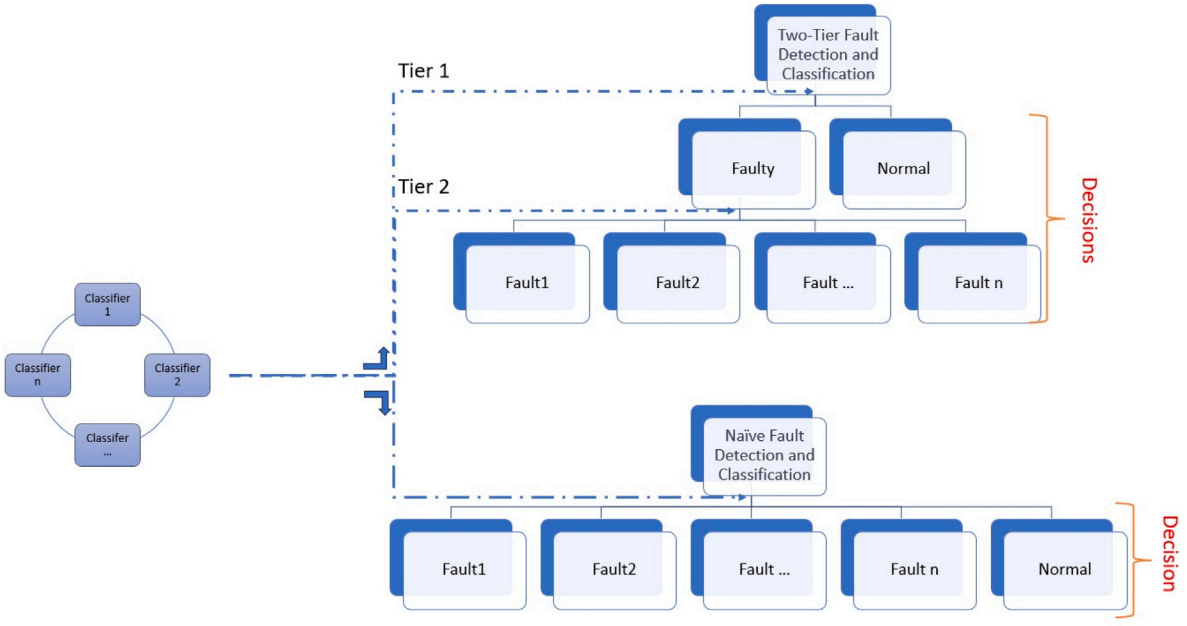


Fig. 6. The naive and proposed two-tier approaches.

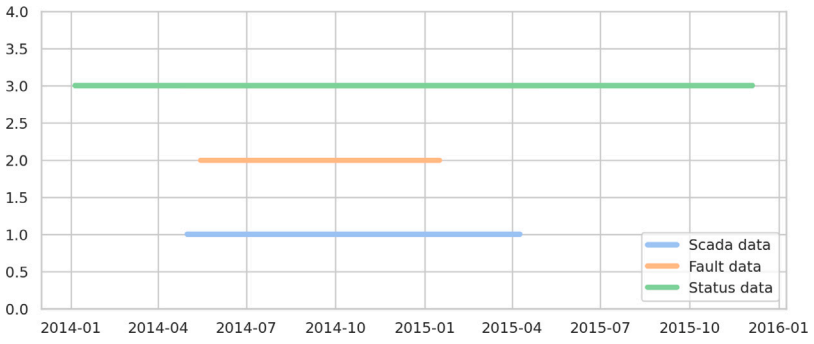


Fig. 7. Temporal extension of the datasets.

Table 6
Wind turbine fault types and descriptions.

Fault type	Description
Generator Heating Fault (GF)	Generator heating faults in wind turbines refer to issues where the generator experiences excessive heating, affecting its performance. Electrical imbalances, high wind speeds, or generator overloading can cause this. Excessive heating can lead to reduced efficiency and potential damage to the generator. Regular monitoring and maintenance are essential to prevent and address these faults.
Mains Failure Fault (MF)	Mains failure faults occur when there is a disruption in the main power supply to the wind turbine. Wind turbines use electricity to operate various components, including pitch control systems, monitoring systems, and grid connections. When the main power supply fails, it can result in a loss of control and may necessitate emergency shutdown procedures. Backup power systems or grid redundancy can mitigate the impact of mains failure faults.
Feeding Fault (FF)	Feeding faults pertain to disruptions in the power feeding mechanism of the wind turbine. This can involve issues with cabling, connectors, or the connection to the electrical grid. These faults can lead to power loss and downtime for the turbine. Regular electrical infrastructure inspection and maintenance are crucial to prevent feeding faults.
Air Cooling Fault (AF)	Air cooling faults involve issues with the turbine's air cooling system, which regulates the temperature of critical components, such as the generator or power electronics. When the air cooling system fails, these components can be overheated, reducing efficiency and potentially causing damage. Proper cooling system maintenance and monitoring are essential to prevent air cooling faults.
Excitation Fault (EF)	Excitation faults relate to issues in the excitation system of the wind turbine's generator. The excitation system is responsible for controlling the generator's output voltage. An excitation fault can lead to variations in electrical output and affect the stability of the turbine's electrical grid connection. This fault may result from issues like controller malfunctions or voltage regulator problems. Timely diagnosis and repair are necessary to address excitation faults.

Table 7
Dataset features and description.

Feature	Description	Unit	Dataset
DateTime	Timestamp	–	All
Time	Time of day	–	1, 3
Error	Error indicator	–	1
Windspeed (ava./max./min.)	Wind speed readings	m/s	1
Rotation (ava./max./min.)	Blades rotation status	degrees	1
Power (ava./max./min.)	Power readings	kW	1
Nacel position	Nacelle position	degrees	1
Operating Hours	Total operating hours	hours	1
Production kWh	Energy production	kWh	1
Reactive Power (ava./max./min.)	Reactive power readings	kVAR	1
Available P (wind/tech./force majeure/external)	Available power readings	kW	1
Blade angle A	Blade angle A readings	degrees	1
Inverter cabinet temp. (Sys 1/2, Inverter 1-7)	Cabinet temperature	°C	1
Spinner temp., Bearing temp., Pitch cabinet, Blade temp., etc.	Various component temperatures	°C	1
Ambient temp., Tower temp., Control cabinet, Transformer, RTU	Ambient and component temperatures	°C	1
RTU: ava. Setpoint 1	RTU setpoint	–	1
Inverter averages, std dev	Inverter statistics	–	1
Main Status	Main system status	–	2
Sub Status	Subsystem status	–	2
Full Status	Comprehensive status	–	2
Status Text	Textual status description	–	2
T, Service, FaultMsg, Value0	Status-related data	–	2
Fault	Fault indicator	–	3

within the fault data frame as were classified as non-faulty instances. Moreover, categorical attributes were translated into numerical values using a label encoder to facilitate analysis. The entire preprocessing procedure included:

- **Temporal Indexation**: All the separated datasets were merged using temporal indexation.
- **Missing Data**: Missingness in the dataset was targeted through interpolation.
- **Label Encoding**: Transforming categorical attributes into numerical representations.
- **Data Balancing**: Using SMOTE (Synthetic Minority Over-sampling Technique) to address the class imbalance in ML. It generates synthetic samples for the minority class by interpolating between existing instances and their neighbors. This rebalancing enhances model performance, reduces bias, and prevents overfitting. However, careful parameter tuning and evaluation are needed to avoid noise and ensure effective results.
- **Feature Selection**: To identify and retain relevant attributes for each model. A feature-averaging technique based on tree models was developed. This technique involves training an ensemble of five powerful tree-based algorithms to predict the target variable. A threshold of 0.1 was set to selectively include features with an average importance above this value. Consequently, a calibrated data frame was obtained, comprising a minimum of 31 features for the binary fault detection problem, 32 features for fault identification, and 30 features for the naive solution encompassing both fault and non-faulty samples. Figs. 8 and 9 depict the obtained results, and Table 8 lists the 32 chosen features.

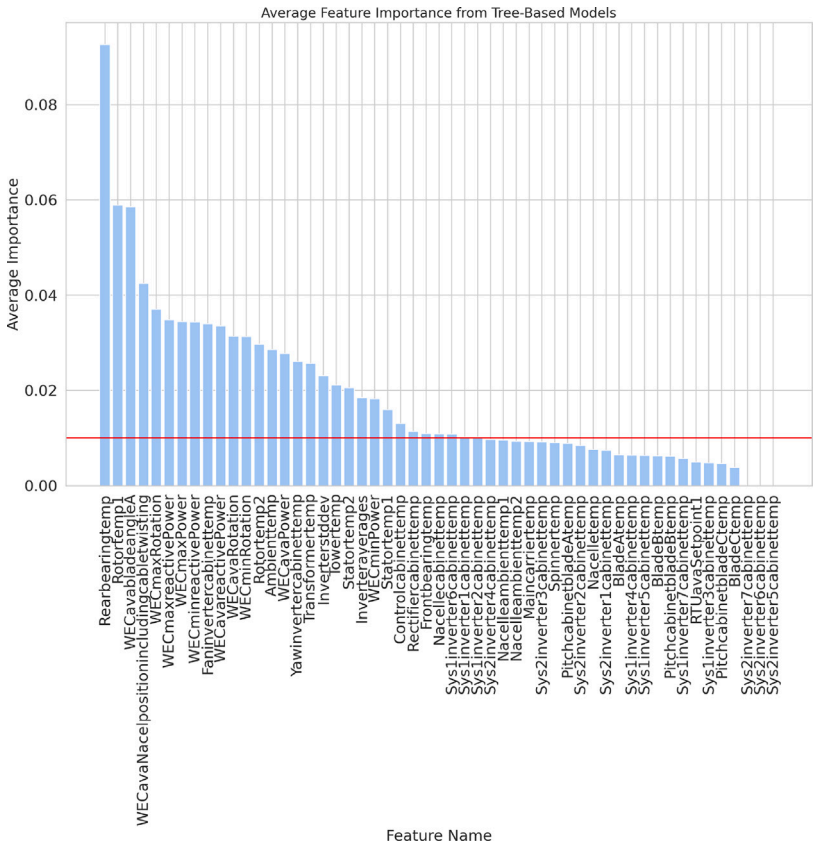


Fig. 8. Naive solution average importance results.

Table 8
Features selected by the averaging technique.

Important features	Important features
WECavaRotation	Spinnertemp
WECmaxRotation	Frontbearingtemp
WECminRotation	Rearbearingtemp
WECavaPower	PitchcabinetbladeAtemp
WECmaxPower	PitchcabinetbladeBtemp
WECminPower	PitchcabinetbladeCtemp
WECavaNacelpositionincludingcabletwisting	BladeAtemp
WECavareactivePower	BladeBtemp
WECmaxreactivePower	BladeCtemp
WECminreactivePower	Nacelletemp
WECavabladeangleA	Nacellecabinettemp
Sys1inverter1cabinettemp	Maincarriertemp
Sys1inverter2cabinettemp	Yawinvertercabinettemp
Sys1inverter3cabinettemp	Faninvertercabinettemp
Sys1inverter4cabinettemp	Rectifiercabinettemp
Sys1inverter5cabinettemp	Nacelleambienttemp1

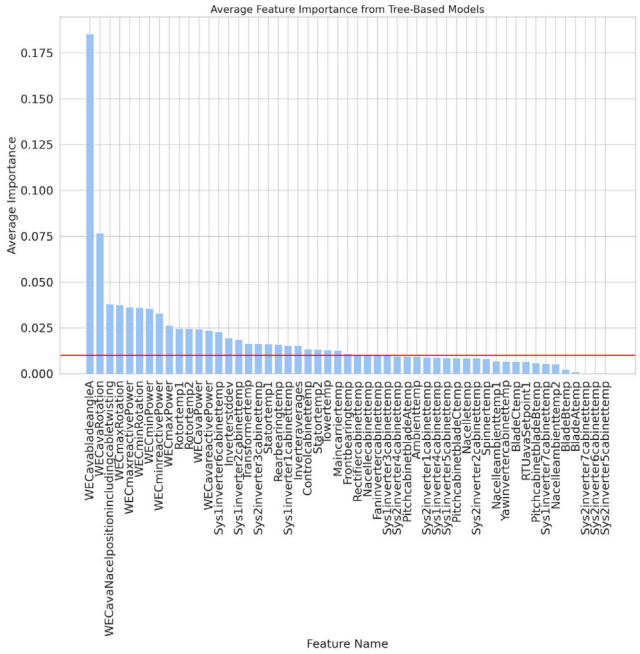
- **Correlation Analysis**: To identify the highly correlated features with the target and the mutually correlated features.
- **Data Scaling**: To ensure uniform influence. Data were scaled to the appropriate range using the MinMaxScaler.

It is worth noting that the effective handling of the imbalanced situation, facilitated by the SMOTE algorithm, entails the integration of proximate data points with the original ones. This strategic approach ensures that the original data's inherent distribution and characteristic attributes are retained, protecting the integrity of the decision-making process within the system. Fig. 10 illustrates this process through bar plots, showcasing the resulting training data distribution post-application of the SMOTE technique.

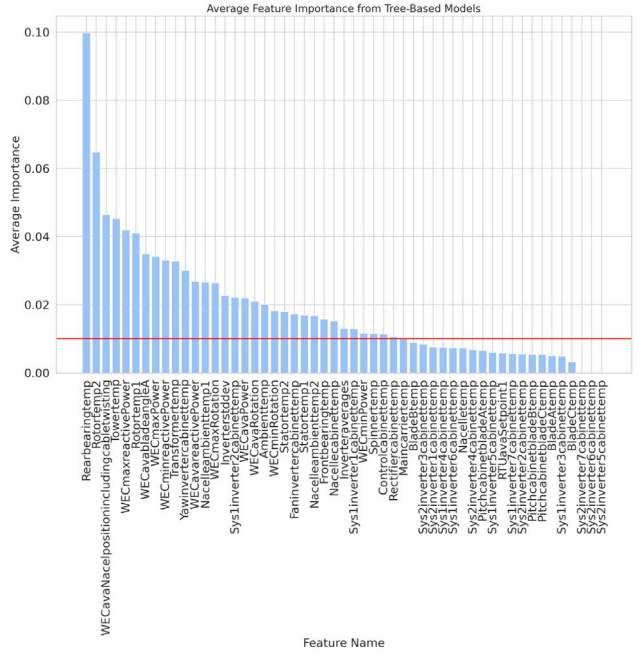
5.4. Exploratory data analysis

Initially, a comprehensive analysis focused on the electricity produced by the wind energy converter, representing the wind turbine's generation mechanism. This analysis is significant as it can potentially provide insight into the occurrence of anomalies during typical operations. A preliminary resampling procedure was applied to the max-generated power feature and the dataset. Interestingly, there is a discernible correlation between generated power and fluctuations in wind intensity. In particular, instances where generated power deviates while wind speed remains relatively stable could signify aberrant behavior or faulty conditions. These assumptions are also driven by the fact that there is a strong linearity between these two characteristics, as depicted in Fig. 11.

Gaining insights into the fault distribution within the dataset is a significant endeavor, which prompted the creation of a histogram showcasing the proportion of fault occurrences. The analysis in Fig. 12(a) reveals a notable presence of non-faulty instances, constituting over 60% of the samples, while various fault types collectively account for approximately 36%. This scenario underscores the apparent imbalance in the distribution of faults within the dataset, potentially causing a model to exhibit a bias towards the majority class rather than the minority. Moreover, comprehending the manifestation of faults and identifying their temporal occurrence is paramount. To this end, a bar plot depicting the occurrences of faults over the considered period is presented in Fig. 12(b). Interestingly, higher frequencies of faults are discernible during October, November, and January, contrasting with other months. This correlation gains further significance when considering that these months coincide with elevated wind speeds. This correlation potentially indicates a link between fault occurrences and external factors, such as wind turbine stress or rotational frequency, resulting from increased wind speeds during these months. Furthermore, it is apparent that distinct categories of faults manifest during particular



(a) Binary fault detection step: average importance results



(b) Multi-class fault identification step: average importance results

Fig. 9. Tree-based averaging importance technique results for the proposed framework.

months: GF is prevalent in May, AF emerges in September, and FF surfaces in December. While explicit causality between these faults and their respective months is not widely documented in existing literature, it is conceivable to deduce a tentative connection. Indeed, variations in environmental factors during these months might plausibly contribute to the occurrence of these faults. Consequently, prudent maintenance strategies could be formulated based on these patterns, entailing the scheduling of maintenance operations aligned exclusively with these specific fault types. This approach to decision-making can optimize the efficiency of wind turbine systems and enhance their overall reliability.

As previously described, three distinct datasets encapsulate the operational performance of the wind turbine. The first dataset pertains to fault warnings, while the second focuses on the current status of the turbine, including textual descriptions related to its operational state. The third dataset encompasses the SCADA data, which comprises various entities. These entities necessitate analysis in conjunction with the fault data. After merging both datasets, the entities were averaged and subsequently examined in correlation with each fault type. Based on the obtained findings, a range of hypotheses can be constructed. Within this array of hypotheses, it is plausible to consider that GF predominantly emerges when temperature values are relatively low. In contrast, the remaining fault types arise when temperature values are elevated compared to standard operational conditions. Upon scrutinizing the thresholds of occurrence, a notable pattern emerges. Almost all fault types are observed to manifest at temperatures exceeding 20 °C, except for GF, which occurs at temperatures below this threshold. Furthermore, an interesting connection becomes apparent when examining operating hours about specific fault types. Specifically, the MF category exhibits a pronounced association with operating hours, often appearing during the initial phase of wind turbine operation, encompassing fewer than 2000 h. In contrast, AF tends to manifest between the 2000 and 3000-hour mark of operation. Other fault types tend to become discernible after the wind turbine has accumulated more than 4000 h of operational time. These data are visualized through box plots in Fig. 13, validating the hypotheses. These observations are crucial in developing an effective fault detection model, paving the way for finely tuned maintenance strategies. For example, when temperatures are below 20 °C, maintenance teams can focus solely on anticipating and addressing GF, minimizing the resources and personnel required for intervention. Additionally, the urgency of attending to AF becomes apparent, particularly within the initial operational span of fewer than 2000 h. Intervention is imperative in this timeframe. Conversely, a distinct pattern emerges for other fault categories. Instances where component temperatures surpass the 20 °C threshold, coupled with operating hours exceeding 2000, indicate potential fault occurrences. When these insights are considered, an optimized maintenance protocol can be meticulously designed, ensuring that the allocation of resources and interventions are fine-tuned to specific fault types, temperature conditions, and operational hours. This strategic approach has the potential to substantially improve the efficiency and reliability of the wind turbine system while mitigating downtime and optimizing resource utilization.

6. Results and discussion

The training process for all models was conducted using Google Colab, a cloud-hosted platform by Google designed for Python code execution. This platform provides valuable resources such as GPUs and TPUs, which significantly enhance the speed of computations and are particularly beneficial for ML, DL, and data analysis tasks. Google Colab also facilitates collaborative work through Jupyter Notebooks.

The dataset was divided into training and testing subsets, with 80% allocated for training and 20% for testing. Following this, essential preprocessing steps were applied to ensure data quality and prepare it for modeling. Subsequently, classifiers were tested according to the nature of the task, whether it involved the naive or two-tier approaches. This systematic approach ensured the models were trained and evaluated effectively before deployment or further refinement. The results are presented in the following order:

- **Results of the Proposed Two-Tier Approach**
- **Results of the Naive Approach**
- **Tuning and Postprocessing Sensitivity: Towards a Better Solution**
- **Comparison with Existing Solutions**

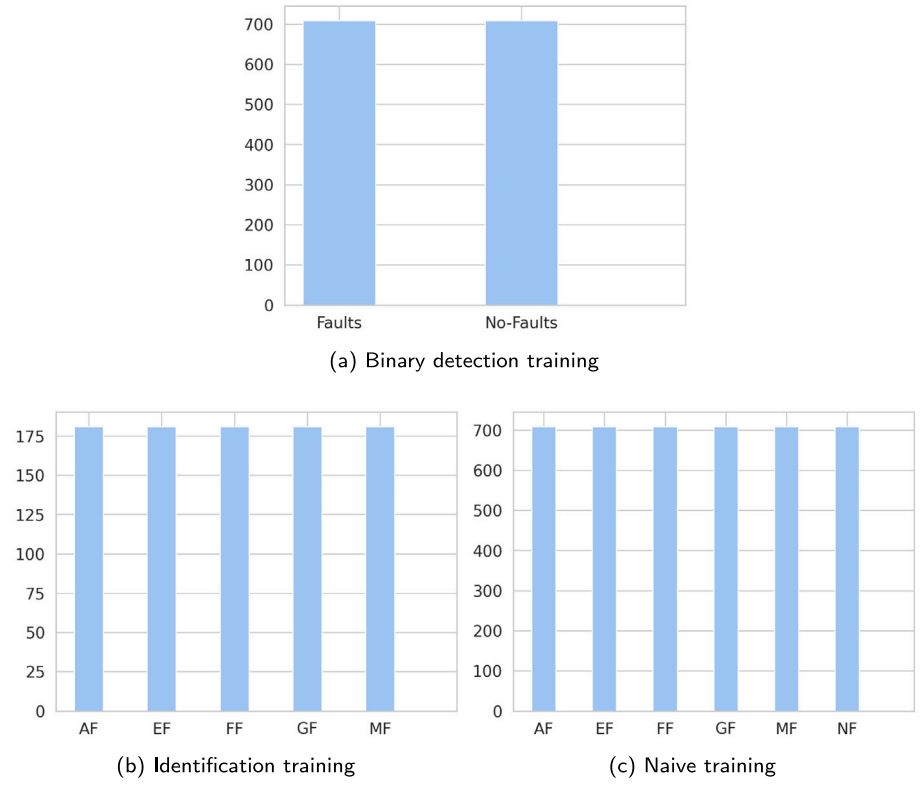


Fig. 10. SMOTE results on training sets for the (a)–(b) two-tier approach and (c) the naive approach.

Table 9
Tier 1 of the proposed solution: binary classification (Fault/No-Fault).

Algorithm	False positive	False negative	Accuracy	Precision	Recall	F1-score	AUC
Bagging	**0.058**	**0.034**	**0.954**	**0.954**	**0.954**	**0.954**	**0.954**
Voting	0.062	0.034	0.952	0.952	0.952	0.952	0.952
CatBoost	0.069	0.034	0.948	0.949	0.9	0.949	
HistGradientBoosting	0.069	0.045	0.943	0.944	0.943	0.943	0.944
RF	0.079	0.034	0.943	0.944	0.943	0.943	0.944
XGB	0.079	0.034	0.943	0.944	0.943	0.943	0.944
LIGHT	0.076	0.041	0.942	0.942	0.942	0.942	0.942
Stacking	0.076	0.045	0.940	0.940	0.940	0.940	0.940
ET	0.093	0.027	0.940	0.942	0.940	0.940	0.942
MLP	0.055	0.072	0.936	0.937	0.936	0.936	0.937
GBoost	0.100	0.038	0.931	0.933	0.931	0.931	0.933
Adaboost	0.120	0.079	0.900	0.901	0.900	0.900	0.901
DT	0.134	0.076	0.895	0.897	0.895	0.895	0.897

6.1. Results of the proposed two-tier approach

The proposed approach comprises two distinct tiers. In the initial tier, a classification task is undertaken to differentiate between faulty and non-faulty samples, utilizing the previously provided classifier. This tier involves a binary classification problem marked by clear-cut boundaries where a dichotomous distinction guides decisions. Subsequently, the second tier focuses on treating faulty samples in isolation, excluding non-faulty instances. Here, the classifiers are tasked with identifying and diagnosing the specific fault type. Notably, this technique adopts a noteworthy strategy by initially addressing the binary classification, which offers less complexity than the multiclass classification. Subsequently, it employs the multiclass classification technique, albeit with fewer classes – five in total – compared to the six delineated in the initial naive solution. This approach is distinguished by its systematic progression from a more straightforward binary classification to a more targeted multiclass classification, with the intricacies of the problem being handled in a strategically sequenced manner.

Tables 9 and 10 give the results of the two tiers. The bagging classifier stands out as the most effective model. For binary classification,

Table 10
Tier 2 of the proposed solution: multiclass classification (AF, EF, FF, GF, MF).

Algorithm	Accuracy	Precision	Recall	F1-score
Stacking	**0.847**	**0.860**	**0.847**	**0.845**
RF	0.795	0.799	0.795	0.796
CatBoost	0.784	0.790	0.784	0.786
ET	0.784	0.789	0.784	0.785
Bagging	**0.778**	**0.786**	**0.778**	**0.780**
Voting	0.778	0.783	0.778	0.780
XGB	0.775	0.780	0.775	0.777
LIGHT	0.764	0.765	0.764	0.765
HistGradientBoosting	0.762	0.765	0.762	0.763
GBoost	0.762	0.766	0.762	0.763
MLP	0.734	0.747	0.734	0.733
DT	0.721	0.729	0.721	0.724
Adaboost	0.356	0.390	0.356	0.283
Tuned Stacking	**0.909**	**0.911**	**0.908**	**0.908**

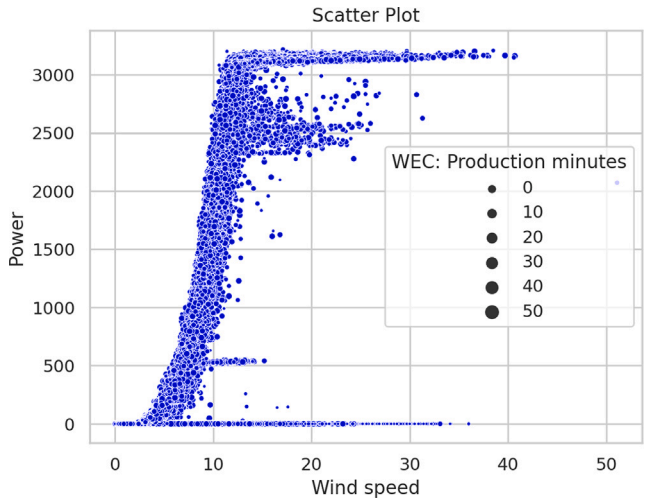


Fig. 11. Scatter plot of wind speed and wind generated maximum power.

bagging surpassed the threshold of 95% in accuracy, precision, recall, and F1-score. It is closely followed by the voting classifier with the second-best performance and then CatBoost. Using CatBoost as the base estimator for the bagging classifier is motivated by its consistent and reliable outcomes. Regardless of the specific task, CatBoost consistently delivers high precision, recall, accuracy, and F1-score. Additionally, analysis of false positive and false negative rates confirms their relatively low values. Furthermore, the substantial AUC value of 95% underscores the bagging estimator's strong generalization capability and discriminative ability, indicating its effectiveness in distinguishing between faulty and non-faulty cases. When examining the outcomes of the multiclass classification task for fault identification, explicitly focusing on the faulty samples, it becomes evident that the bagging classifier's performance is less satisfactory. Its precision, recall, accuracy, and F1-score do not surpass the 78% threshold, which is remarkably similar to the results observed in the basic solution. This reaffirms the assumption that the bagging classifier encounters challenges when dealing with certain fault types. Conversely, the stacking ensemble model emerges as the most effective choice for fault identification. It shows an overall precision of 86%, accompanied by accuracy, recall, and F1-score that surpass 84%. Following closely are the random forest model and the CatBoost model. The bagging classifier in the initial tier demonstrated a swift training time of 0.03 s across the entire dataset, indicating efficient analysis of all samples. Furthermore, a learning curve analysis was conducted at this stage in Fig. 14(a). While it is apparent that maximum accuracy in the validation set is attained in the later stages, the curve indicates early convergence towards a result surpassing 90% accuracy after analyzing 600 samples. The training accuracy remains stable at 100%, suggesting effective capture of underlying patterns within the training set. The model's generalizability was confirmed through validation score results, boasting an accuracy exceeding 95.5%. Similarly, the stacking ensemble learner utilized for the second tier of fault identification exhibited a training time of 15 s across the entire dataset, which is typical given the composition of this learner, particularly with the CatBoost framework. Additionally, the learning curve obtained for the testing set, as depicted in Fig. 14(b), reveals that the model's performance ranges from 50% to 80% at early stages and is characterized by fluctuations. However, after analyzing 250 samples, the model begins to converge towards the optimal solution, achieving 90% accuracy. Furthermore, the model's generalization ability is evident, as its testing set accuracy surpasses its training set accuracy after analyzing approximately 120 samples. This, coupled with the observations mentioned previously, demonstrates that the model is not solely reliant on the training set or a specific dataset; rather, it performs well even when its training is not at a desirable level.

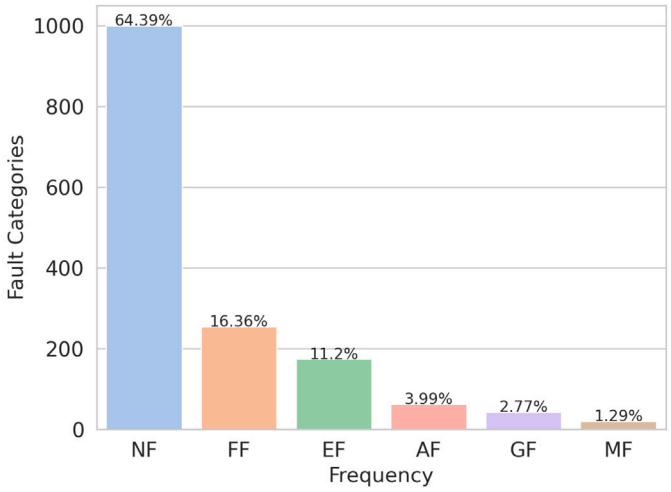
Table 11
Performance metrics of algorithms (naive approach).

Algorithm	Accuracy	Precision	Recall	F1-score
Bagging	**0.787**	**0.792**	**0.787**	**0.787**
GBoost	0.782	0.787	0.782	0.783
CatBoost	0.780	0.783	0.780	0.780
RF	0.775	0.784	0.775	0.775
XGB	0.769	0.774	0.769	0.768
LIGHT	0.767	0.771	0.767	0.767
Voting	0.767	0.772	0.767	0.767
HistGradientBoosting	0.748	0.753	0.748	0.746
Stacking	0.721	0.733	0.721	0.718
ET	0.691	0.714	0.691	0.688
MLP	0.683	0.691	0.683	0.680
DT	0.663	0.675	0.663	0.660
Adaboost	0.473	0.531	0.473	0.463

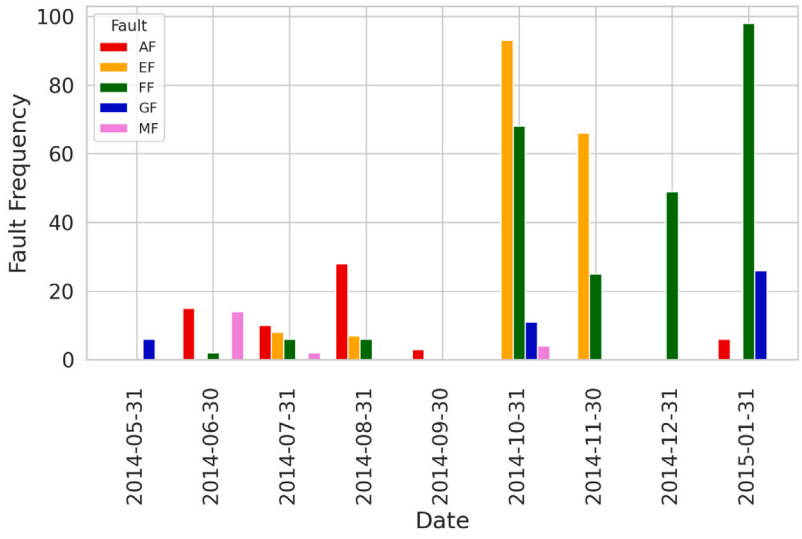
6.2. Results of the naive approach

The naive approach predicts instances of non-faulty occurrences and directly identifies faults within the datasets. A concise evaluation of 13 classifiers addressed tree-based classifiers (such as decision trees, random forests, XGBoost, LightGBM, CatBoost, and Extra Trees Classifier). Boosting classifiers, namely Gradient Boosting and AdaBoost, were also included, along with a feed-forward MLP neural network featuring a single hidden layer. Ensemble learners, specifically bagging, voting, and stacking, were utilized. The objective of employing an extensive training loop encompassing a diverse set of classifiers is to encompass all feasible classifier variations and to facilitate a direct comparison with similar implementations from prior research endeavors. This approach allows for a comprehensive assessment of the performance of various classifiers, ensuring a thorough evaluation against analogous methodologies employed in earlier studies. The effectiveness of these models was evaluated using four metrics that measure overall performance. The results of this evaluation are summarized in Table 11. The superior model in terms of performance emerged as the ensemble learner using the CatBoost framework as the fundamental estimator, employing 15 instances. Following closely was the gradient boosting framework, followed by the CatBoost model. Notably, this ensemble learner, based on bagging, achieved the highest accuracy, precision, recall, and F1-score, reaching approximately 79%. It is worth highlighting that the prominence of the bagging ensemble model is of significant importance. Bagging ensembles exhibit a capacity to mitigate overfitting by training diverse classifier instances on distinct data subsets. Furthermore, introducing variability through the random sampling technique is especially pivotal, particularly in wind energy fault detection. This domain is characterized by multifaceted variability across different elements, necessitating a model's adeptness at accommodating pattern shifts.

Fig. 15 depicts the confusion matrix associated with the bagging classifier where accurate classifications are evident for non-faulty instances, MF, and GF fault types, with recall rates exceeding 91%. However, the classifier encountered challenges in effectively identifying the EF, FF, and AF fault types, exhibiting recall rates below 71%. This disparity in performance gives rise to a plausible assumption. Specifically, distinguishing GF becomes feasible by utilizing operational hours and temperature of specific components, as depicted in Fig. 13. Similarly, detecting MF is achievable only when the operational hours are considered. Yet, these assumptions do not hold for FF, AF, and EF faults. In these cases, the decision boundaries are constrained, and the values of pivotal parameters, including those mentioned, closely resemble those of non-faulty instances. Consequently, the classification system encounters increased difficulty distinguishing between these fault categories and the non-fault instances.



(a) Distribution of faults where NF stands for No Fault



(b) Fault frequency versus time

Fig. 12. Analysis of fault distribution.

6.3. Comparison

The importance of this two-tier approach lies in its departure from the conventional approach of employing a single model for the entire task, which can result in a trade-off between its effectiveness in one aspect versus another. Instead, this system capitalizes on the strengths of one model for fault detection and another for fault identification. By doing so, an approximate average accuracy of 90% can be achieved, surpassing the precision of the basic framework. Moreover, this approach employs a problem-decomposition methodology, unlocking the potential to test numerous solutions in both tiers. This allows for a wide array of possible combinations, and the pipeline can always be constructed using the best-performing models for the first and second tiers. This strategy optimizes the utilization of available resources and enhances the system's overall performance. To delve deeper into the impact of the two-tier approach, a comparison was conducted between the confusion matrices of both approaches. Specifically, the most proficient bagging ensemble learner was considered for this analysis. Figs. 16, 17, and 18 illustrate this comparative study. After thoroughly examining these matrices, it is evident that the best model within the naive approach (see Fig. 16), faces challenges in classifying 7% of the non-faulty cases. In contrast, it demonstrates impressive performance in detecting MF and GF faults, achieving results above 91%. However, diagnosing FF, EF, and AF faults appears to be more intricate, with success rates of 56%, 65%, and 71%, respectively. In this multiclass scenario, the direct classification approach is complex for even the most proficient classifiers, such as the bagging model. The primary reason for this complexity is the number of classes involved. In a scenario with six distinct fault classes, it becomes challenging for the model to distinguish between them, particularly when these faults exhibit similar behavior or when the feature values are closely aligned. In an alternative scenario, the process is divided into two tiers. The first tier involves determining whether a sample is faulty or non-faulty. Subsequently, the system diagnoses the specific fault type for each faulty sample. As illustrated in the corresponding matrix (see Fig. 17), after fine-tuning the hyperparameters of the bagging ensemble, which proved to be the best learner in this tier, fault detection achieves an impressive 95% accuracy in correctly identifying faulty instances. These correctly identified faulty samples are then passed on to the second tier of fault detection, as shown in Fig. 18. It is readily apparent that the number of instances correctly classified as FF, EF, and AF faults has improved compared to the naive approach, achieving respective

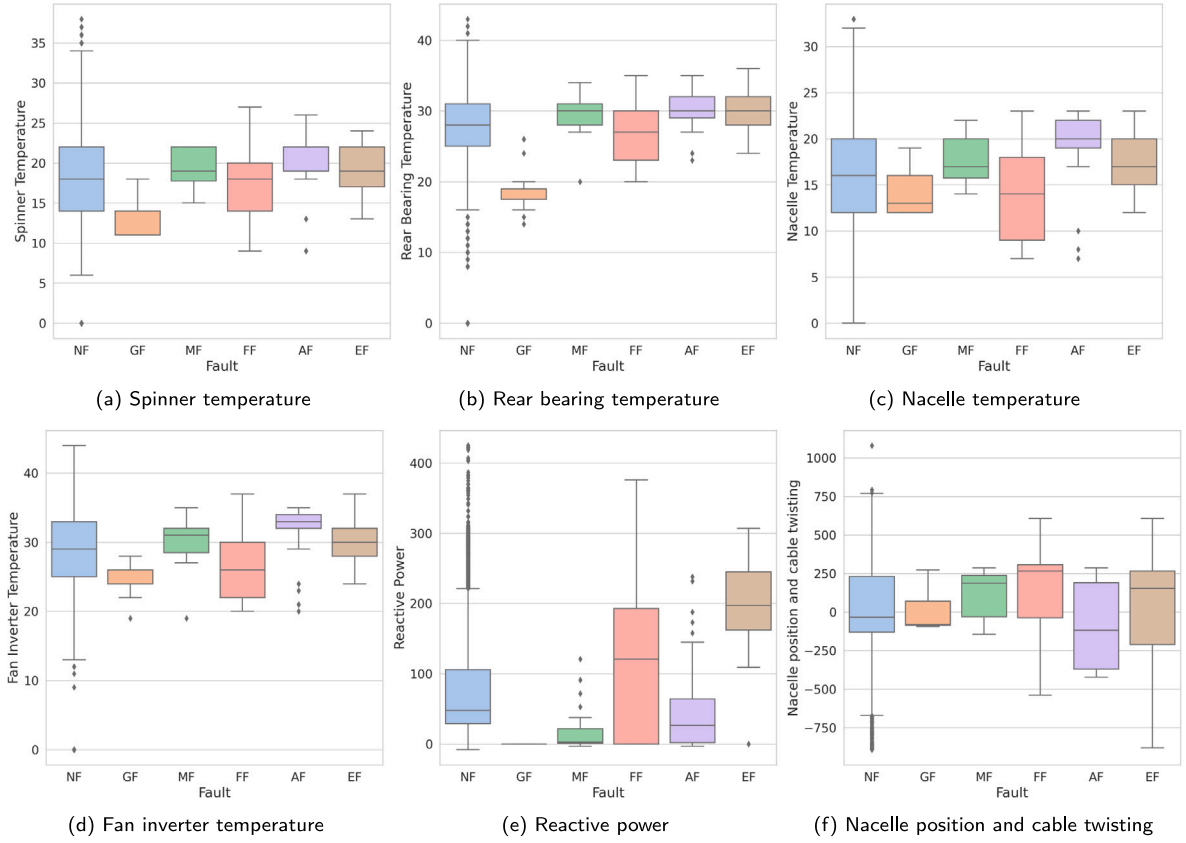


Fig. 13. Exploration (box plot) of fault types across various features: (a) spinner temperature, (b) rear bearing temperature, (c) nacelle temperature, (d) fan inverter temperature, (e) reactive power, (f) nacelle position and cable twisting.

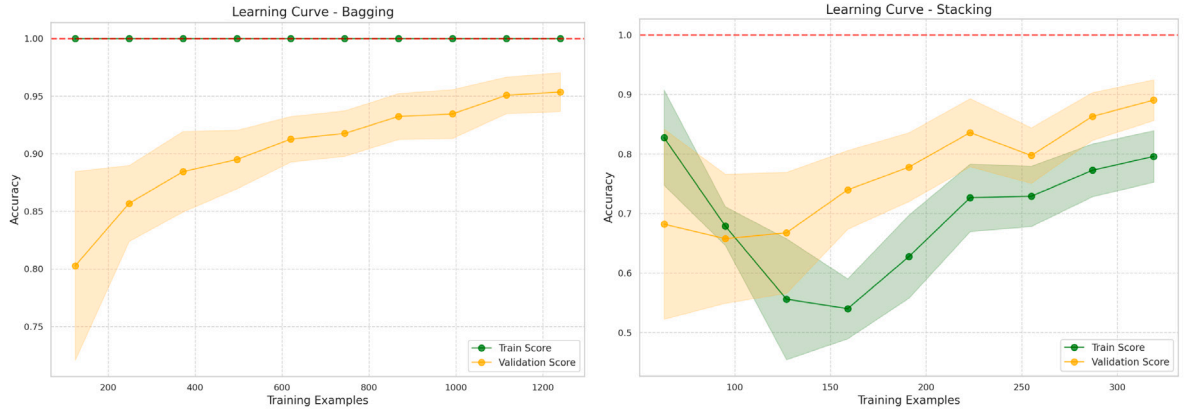
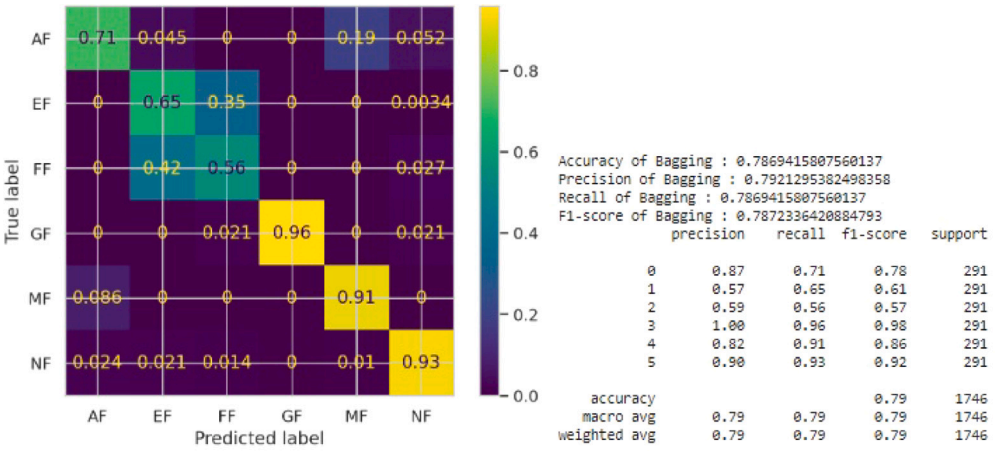


Fig. 14. Learning curves of the models in the two-tier approach: (a) bagging and (b) stacking.

accuracies of 84%, 97%, and 84% (compared to 56%, 65%, and 71% in the naive approach). This demonstrates that the second scenario significantly enhances the results obtained in fault identification.

To gain a deeper understanding of the key factors influencing the models' decisions, a novel feature importance technique was employed. It involves aggregating the values of the importance of features between all classifiers. The objective was not solely to identify determinative features for an individual model but to extract significant insights into the holistic decision-making performance of the entire system. This approach made the shared significant features for the predictions the most impactful. Consequently, these features play a crucial role in delineating the factors that should be considered when designing maintenance strategies. The bar graphs in Fig. 19 depict the critical analysis of features for direct classification (naive approach), the first tier (binary classification), and the second tier (fault identification). As the basic model did not perform well in detecting faults, it became necessary to delve into the significant features of this situation and explore the proposed solution. In the initial scenario, key influencers were rotor temp1, WEC avablade angleA, and nacelle position with twisting. In the two-tier process, models leaned on WEC avablade angle, WEC windspeed, and minimum power for detecting faults. In contrast, they relied on rotor temp1, rear-bearing temp, and rotor temp2 for diagnosing faults. This is logical, as the first-tier (fault detection) model aims to quickly decide if the sample is faulty or not in contrast to the naive model that aims to distinguish the fault type from the normal case, thus making the problem more complicated. The



(a) Confusion matrix (b) Confusion results

Fig. 15. The confusion matrix of the bagging model, the best classifier within the naive solution.

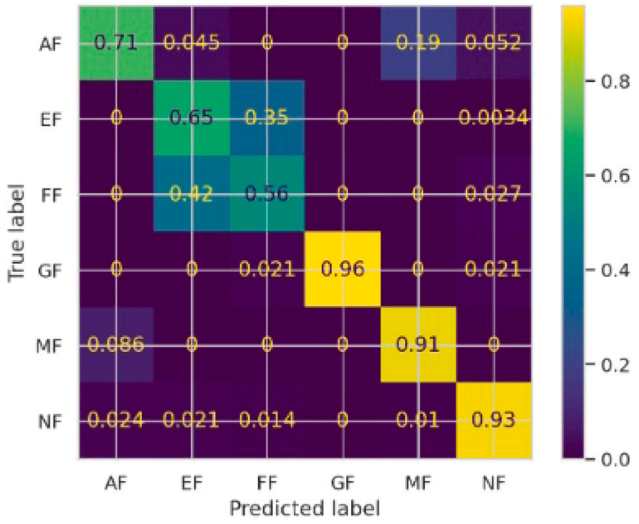
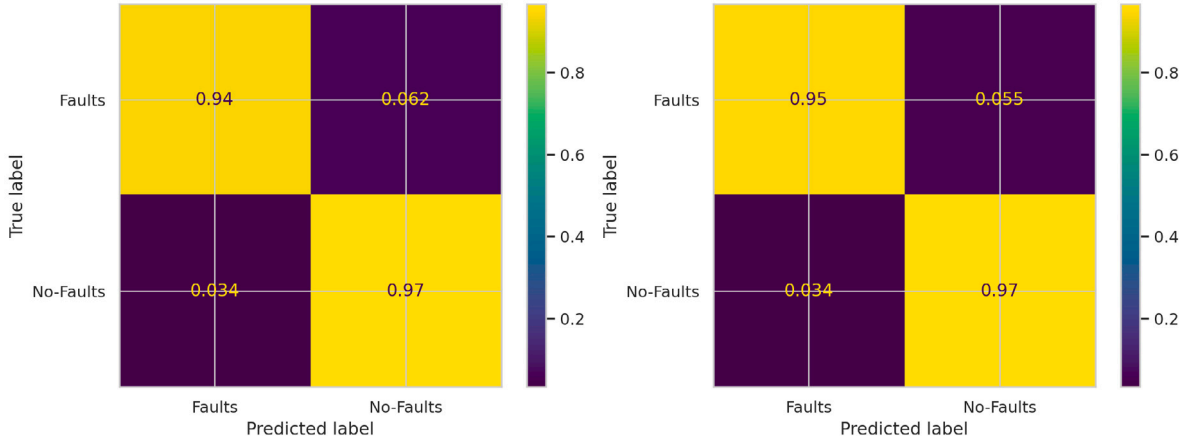
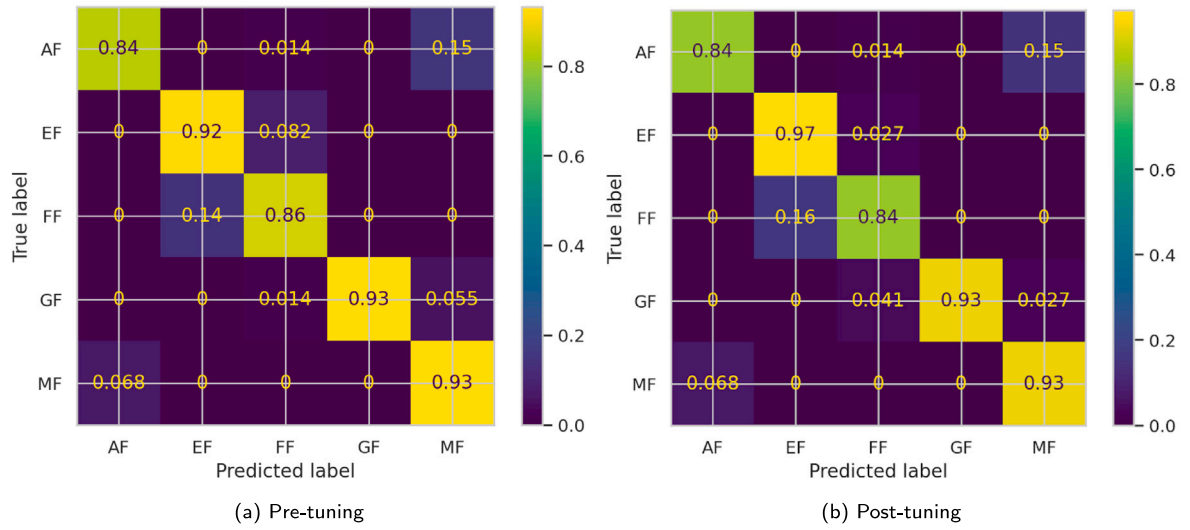


Fig. 16. Naive method (proposed bagging model).



(a) Pre-tuning (b) Post-tuning

Fig. 17. First tier: fault detection with bagging ensemble learner.



(a) Pre-tuning (b) Post-tuning

Fig. 18. Second tier: fault identification with stacking ensemble learner.

proposed solution is different, with unique features for detection and identification. This complexity in the first tier is an advantage for the proposed solution, offering a simpler model.

6.4. Tuning and post-processing sensitivity: Towards a better solution

Fine-tuning hyperparameters plays a pivotal role in significantly enhancing fault detection accuracy during the initial tier. Notably, the correctly classified instances achieved an impressive recall rate of 95%, leading to a notable reduction in misclassified instances. Moreover, the outcomes of fault identification in the second tier, specifically for MF, displayed a slight but noticeable improvement, reaching an accuracy of 93%. This improvement is worth highlighting, even though a minor decrease was observed in diagnosing EF faults. Indeed, the tuning operation brought to light a trade-off between diagnosing different types of faults, especially when considering EF, FF, and MF faults. The proposed approach represents a robust solution for concurrent fault detection and identification. Specifically, fine-tuning the stacking model hyperparameters involved optimizing the composition of its underlying models to enhance overall performance significantly. These refined classifiers were subsequently integrated into the proposed pipeline. The composition of the tuned stacking learner included classifiers such as XGBoost, Random Forest, Extra Trees classifier, and CatBoost. Similarly, a comparable technique was applied to the bagging classifier, constituting the proposed solution's initial layer. The confusion matrices for the fine-tuned bagging and stacking ensembles are presented in Figs. 17 and 18. The analysis focused on the sensitivity of the bagging ensemble model concerning the composing learner. The selection of this learner and the quantity of estimators exert a notable influence on the outcomes. Nevertheless, the results demonstrate a close similarity in terms of accuracy and F1-score. Table 12 presents the sensitivity analysis of selecting the composing model for the bagging classifier during the fault detection tier. The bagging ensemble learner demonstrates its peak performance when the number of base learners, or estimators, reaches 150 units. This observation is particularly pronounced when employing CatBoost as the baseline estimator, showcasing the remarkable power of this algorithm in enhancing ensemble learning accuracy. Moreover, Table 13 presents the results of postprocessing trials for composing models within the fault identification tier using the stacking ensemble learner. Notably, the model's sensitivity to the composition of base learners is evident, with optimal results achieved when CatBoost, ExtraTrees, and XGBoost classifiers are utilized. Interestingly, across all trials, CatBoost consistently emerged as the preferred final estimator due to its superior performance compared to other estimators.

Table 12
Analysis of the bagging ensemble learner in the fault detection tier: sensitivity to the base learner.

Model	Accuracy	Precision	Recall	F1-score
Bagging (CAT)150	**0.9541**	**0.9543**	**0.9542**	**0.9541**
Bagging (CAT)200	0.936	0.935	0.924	0.929
Bagging (CAT)50	0.936	0.937	0.922	0.929
Bagging (CAT)100	0.936	0.937	0.922	0.929
Bagging (XGB)	0.932	0.931	0.922	0.926
Bagging (Random Forest)	0.932	0.935	0.918	0.925
Bagging (Decision Tree)	0.929	0.928	0.917	0.922
Bagging (LGBM)	0.929	0.928	0.917	0.922
Voting (Soft)	0.929	0.932	0.913	0.922

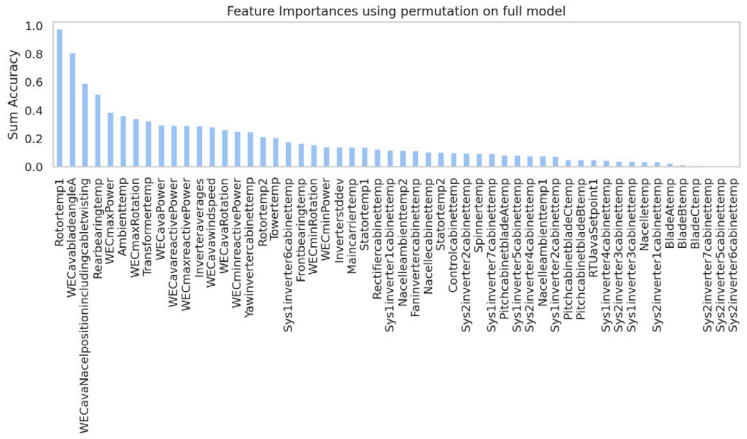
Table 13
Analysis of the stacking ensemble learner in the fault identification tier: sensitivity to the composing learners.

Trial number	Composing models	Accuracy
1	['CatBoost', 'LGBM']	0.8219
2	['CatBoost', 'RF']	0.8329
3	['CatBoost', 'MLP']	0.8301
4	['CatBoost', 'ET']	0.8740
5	['CatBoost', 'ET', 'XGB']	0.8877
6	**['CatBoost', 'ET', 'XGB']**	**0.9096**
7	['CatBoost', 'ET', 'XGB']	0.9014
8	['CatBoost', 'ET', 'XGB']	0.8986

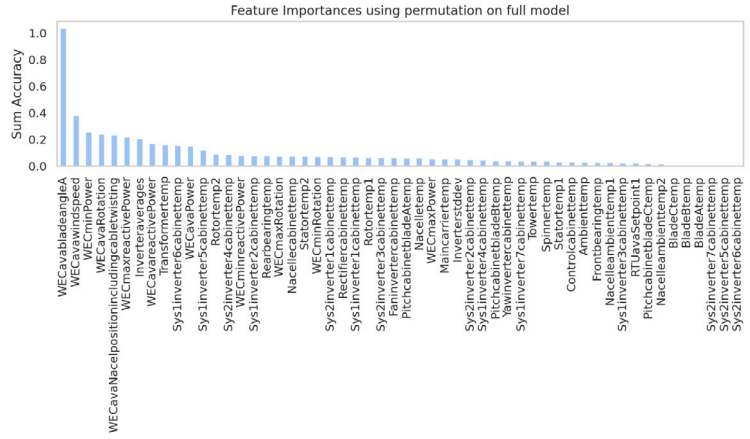
7. Explainability, model validation, advantages, and limitations

To validate the suggested methods, a more comprehensive analysis of the data structure and the embedding of fault samples in their vicinity was conducted using t-SNE visualization in a 2D space. Additionally, explainable artificial intelligence (XAI) ensured the best models' transparency. The following are the findings of this analysis:

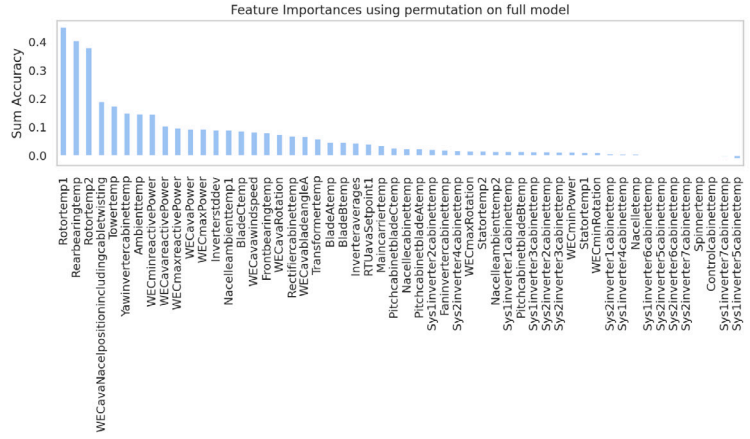
- ***eXplainable AI:***
  XAI tools provide insights into the contributions of various features, facilitating a comprehensive evaluation of model performance. This is particularly valuable in fault detection and identification, as it helps substantiate assumptions about the features responsible for classifying specific anomalies. Notable examples of XAI tools include SHAP and LIME. The SHAP explainer, based on Shapley values from cooperative game theory, assigns a value



(a) Naive model feature importance



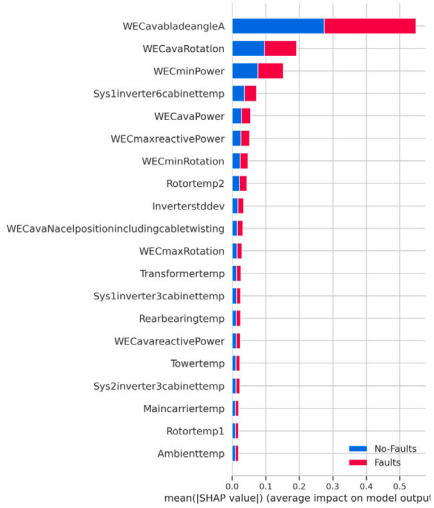
(b) Binary model feature importance



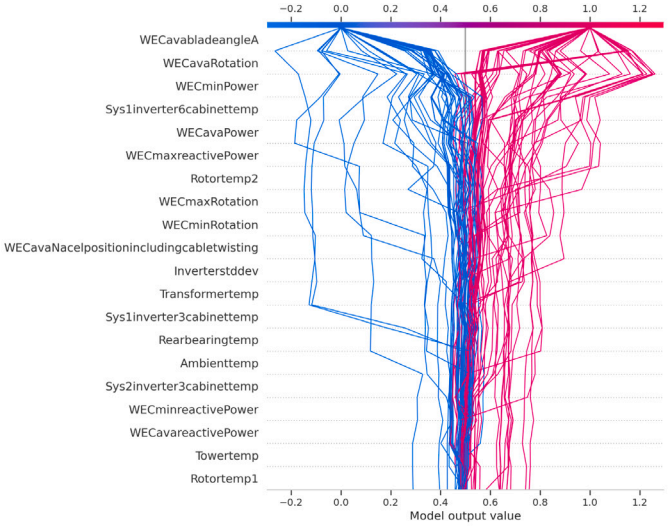
(c) Fault identification feature importance

Fig. 19. Comparison of permutation feature importance.

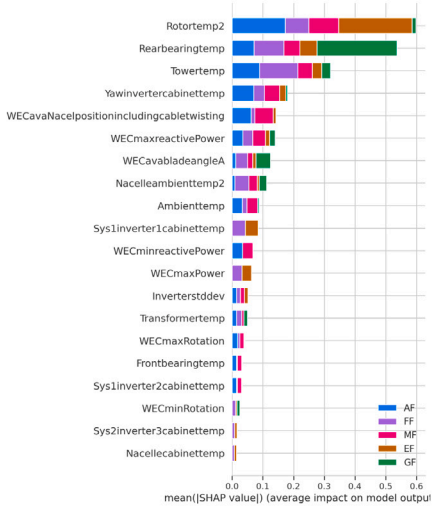
to each feature based on its contribution to the model's predictions. Meanwhile, LIME accurately explains individual predictions by introducing variations to input features and observing resultant changes in the model's output. This process creates a surrogate model that closely approximates the behavior of the opaque model, enhancing understanding of its functioning. The SHAP explainer was initially employed for both the bagging classifier and the stacking classifier in the first and second tiers, respectively. For the bagging classifier and the initial fault detection step, it is evident that certain features play a dominant role in the model's decision-making process. Specifically, features such as ECavabladeangle1, WECavaRotation, WECminPower, Sys1inverter6cabinettemp, WECavaPower, WECmacreactivePower, and WecminRotaion emerge as the primary contributors, significantly influencing the model's output. While other features also contribute to the decision, their impact remain



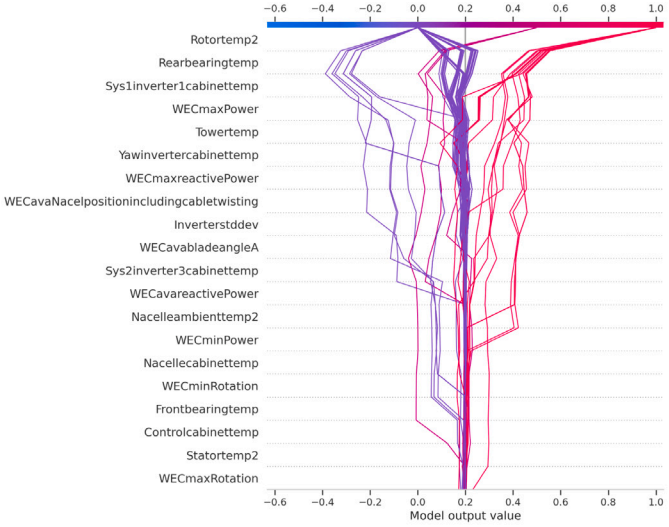
(a) Bagging Classifier First Tier Features contribution



(b) Bagging Classifier First Tier Features Calibration of a given sample



(c) Stacking Classifier Second Tier Features contribution



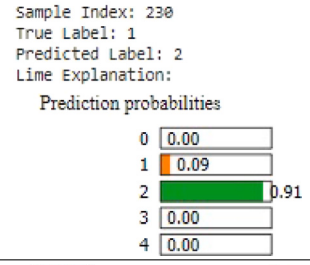
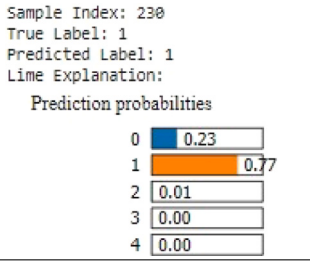
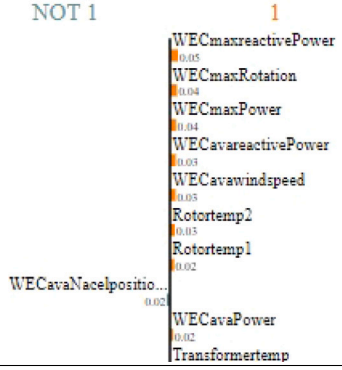
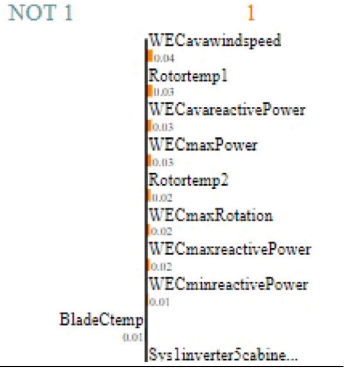
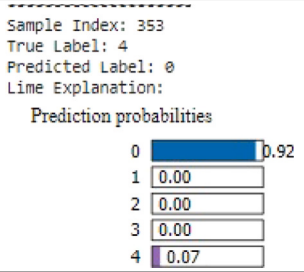
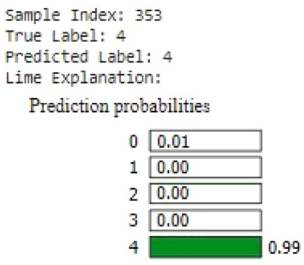
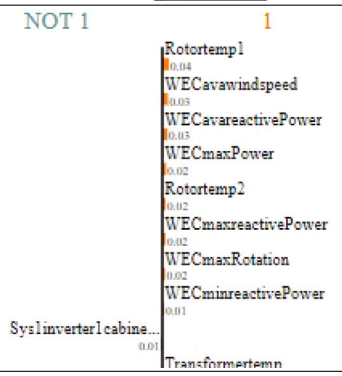
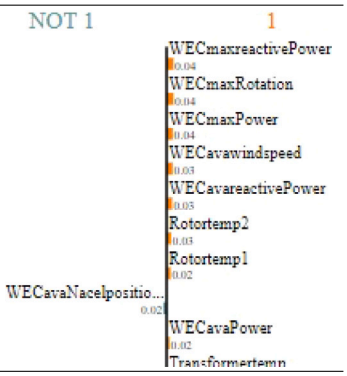
(d) Stacking Classifier Second Tier Features Calibration of a given sample

Fig. 20. Shap explainability of the trained models: Feature contribution and feature interaction for a given sample of the first-tier (fault detection) bagging model (a and b) and the second-tier (fault identification) stacking model (c and d).

below the 0.05 threshold. Furthermore, it is notable that these aforementioned features effectively differentiate between faulty and non-faulty samples, as illustrated in Fig. 20(a). Even the minor contributors among these features play a discernible role, as depicted in Fig. 20(b), where the collective impact of all features, including the lesser contributors, is evident. The initial values of these features are calibrated towards the final decision, incorporating various aspects of calibration. Interestingly, the calibration process does not strictly adhere to the feature importance rankings observed in Fig. 19(b). The model assigns varying degrees of importance to these features at different stages of the decision-making process. While this aligns with the findings from the averaging feature importance technique, it also suggests that a subset of features could potentially yield optimal results, indicating avenues for optimization and feature selection. The analysis was extended to the stacking classifier, revealing from Fig. 20(c) that predominantly six features play the most significant role in diagnosing the specific fault type. These features include Rotor temp2, RearBearingtemp, Towertemp, yawinvertercabinettemp, WECavaNacellposition including cable twisting, WEC max reactive Power, WECavabladeangleA, and Necelleambienttemp2. Although other features also contribute, their impact is comparatively lower. Notably, unlike the other features, the aforementioned features can influence the identification of all five fault types. The extent of their impact and their order vary, as illustrated in Fig. 20(d). Furthermore, the calibration of starting values for a given sample towards the final result is predominantly influenced by a maximum of 10 to 15 features.

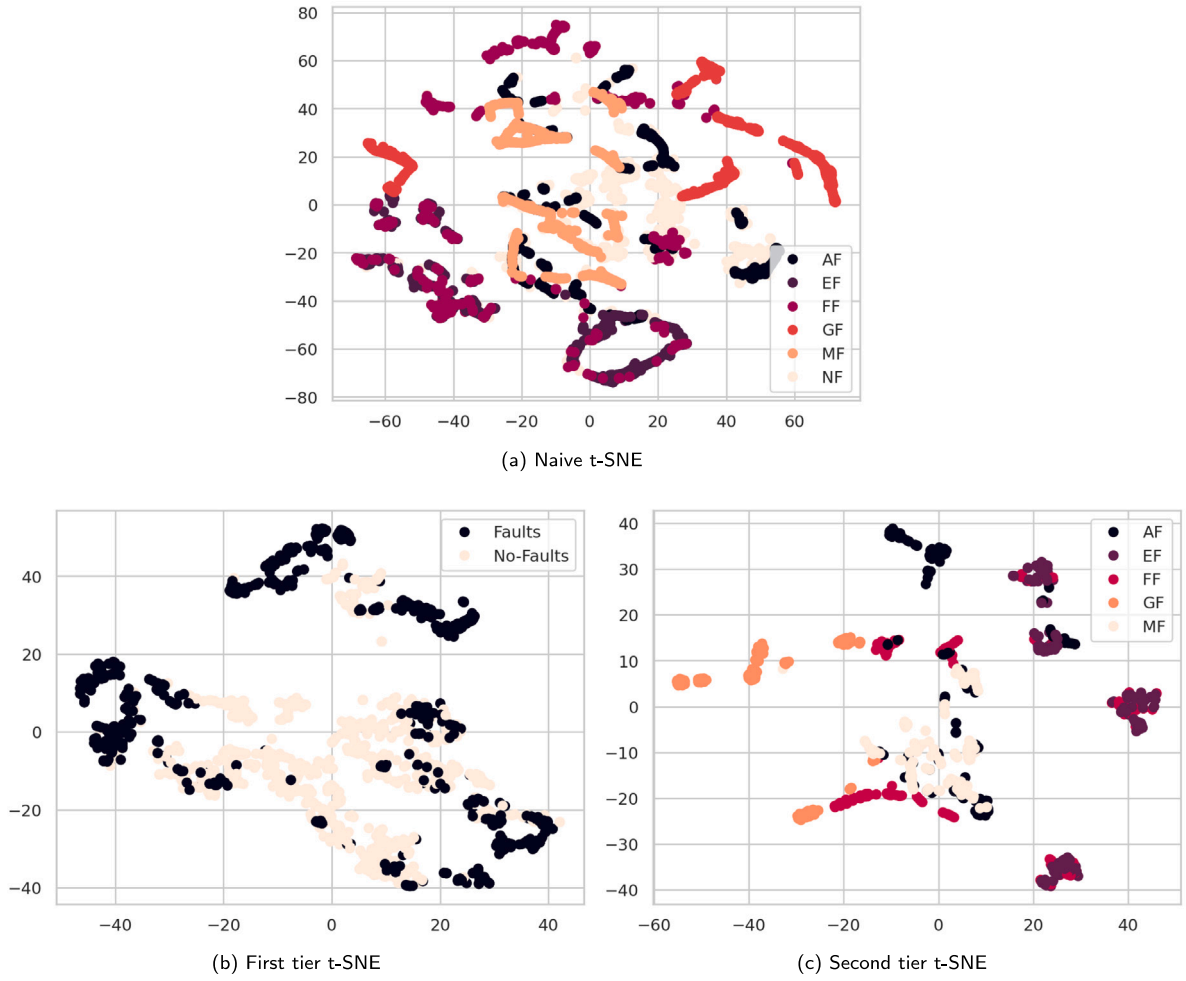
Further investigation was conducted using the LIME analyzer; this time, two samples within the dataset were identified. These samples underwent assessment using the naive bagging approach and the stacking identification tier, as presented in Table 14. It was observed that the naive bagging solution misclassified these two samples. However, upon applying the stacking ensemble learner, the model effectively discerned concealed patterns, resulting in precise classification. This indicates that the proposed two-tier approach can rectify misclassified instances encountered in the

Table 14
LIME explainer results.

Sample index	Bagging Naive solution	Stacking two-tier solution
230		
Features Contribution		
353		
Features Contribution		

naive procedure, a conclusion also applicable to the analysis of the second sample. At this point, a critical examination of the contributing features of both solutions becomes essential. For the 230th sample, it becomes apparent that the bagging-naive classifier heavily relies on attributes such as WEC max reactive power, max rotation, and max power. In contrast, the stacking two-tier learner prioritizes features such as WECavawindspeed, rotor temperature, and WEC reactive power. Importantly, this aligns seamlessly with the prior analysis, confirming the relationship between wind speed and certain fault types. This analysis highlights the superiority of the detection and identification steps over the naive solution, as they leverage the most influential parameters, resulting in more accurate decision-making.

- ***t-SNE Study:*** Using the most impactful features at each stage (fault detection, fault identification), a t-SNE visualization was conducted in a 2D space to assess the contribution of these features in distinguishing between the target classes. Fig. 21 depicts the findings for each stage. The naive visualization in Fig. 21(a) shows that the model encounters difficulties separating the classes, particularly evident with the MF, EF, and GF fault types, which share common values. Additionally, other fault types appear to be more clustered together, while the NF (non-faulty) samples exhibit centralized behavior. Conversely, in the two-tier approach illustrated in Figs. 21(b) and 21(c), the features can effectively differentiate between faulty and non-faulty samples. This enhances the clarity of the second fault identification step, with GF, EF, and MF faults being notably more distinguishable. Other fault types are also classified reasonably well. These observations affirm the superiority of the proposed approach over the naive one and highlight its suitability for handling complex class relationships.



(a) Naive t-SNE

(b) First tier t-SNE

(c) Second tier t-SNE

Fig. 21. Performance comparison of the naive and proposed approaches with 2D t-SNE visualization.

7.1. Advantages and limitations of the current work

The notable advantage of the proposed framework lies in its distinct two-tier approach, a departure from the conventional naive solutions prevalent in the literature. The inherent complexity of classifying in the naive solution requires six classes, encompassing both the non-faulty and the six fault types. In contrast, the proposed approach efficiently reduces this complexity to two classes in fault detection and five classes in fault identification. This reduction is particularly advantageous because ML classifiers generally perform better with lower class complexity. The validation of this advantage is evident in the accuracy values, showcasing superior performance in the two-tier approach compared to the best classifier in the naive solution. Employing ensemble learners in the framework adds another layer of flexibility and breadth to the study. Ensemble learners, composed of both base and meta learners, provide a wide array of options for future research. While the study achieved commendable results with a diverse set of choices, it acknowledges a limitation in not covering all possible combinations for ensemble learners. Additionally, incorporating the LIME analyzer stands out as a strength of the study. The authors assert that any ML-related work should be accompanied by validation through explainable artificial intelligence (XAI) or a suitable tool to enhance transparency and instill trust in ML systems. However, the study acknowledges certain limitations, such as the inability to achieve 100% accuracy in both tiers. The authors suggest that future work addresses this limitation and attain higher accuracy levels.

8. Conclusion

This study addressed the challenge of fault detection and identification in wind energy converters, aiming to develop a solution that balances accuracy and complexity using ML and data analysis methods. The objective was to improve the utilization of wind turbines, reduce maintenance costs through an effective detection system, and facilitate the widespread implementation of these systems. A dataset originating from a wind turbine's SCADA system located in the southwest of Ireland underwent collection, preprocessing, and analysis. Employing a feature importance technique, the dataset dimensionality was reduced from 53 to 32 features. Two distinct scenarios were explored, accounting for the presence of five fault types within the dataset. The first scenario involved a basic approach integrating simultaneous fault detection and identification, encompassing six classes. The second scenario introduced the proposed two-tier solution, delineating separate stages for fault detection and identification, with respective complexities of two and five classes. Evaluation encompassed twelve classifiers, ranging from tree-based models, neural networks, and ensemble learners to traditional ML algorithms like boosting frameworks. The two-tier approach achieved the highest accuracy, with the bagging ensemble learner using the CatBoost framework achieving 95.4% accuracy for fault detection and an ensemble of CatBoost, ExtraTree, and XGBoost achieving approximately 91% accuracy for fault identification. In comparison, the basic approach achieved a maximum accuracy of 78.8% with the bagging ensemble learner. Furthermore, eXplainable Artificial Intelligence (XAI) tools were utilized to validate the chosen features and assess their contribution.

The utilization of ensemble learners in the proposed framework offers several noteworthy advantages. Firstly, it reduces complexity by partitioning the problem into two distinct subproblems: fault detection and fault identification. This segmentation allows for a more targeted approach, where the primary focus of the first tier is solely identifying faulty samples, while the subsequent tier is dedicated to diagnosing specific fault types. The overall complexity is effectively managed by breaking down the problem in this manner, leading to clearer and more efficient analysis processes. Secondly, the ensemble learning scheme employed in the framework demonstrates dynamic adaptability. Unlike static approaches, which may be rigid and less responsive to changes, ensemble learners can incorporate a variety of classifiers, both in terms of type and number. This flexibility enables the framework to evolve and adjust according to the specific requirements of the problem at hand. Additionally, the ensemble learning approach enhances robustness and resilience, as it combines the strengths of multiple classifiers, mitigating the weaknesses inherent in individual models. Furthermore, the proposed framework exhibits potential applicability beyond the initial dataset. The solution can theoretically be extended to datasets from diverse geographic regions by leveraging ensemble learners and the averaging importance technique. This scalability enhances the framework's versatility, making it a valuable tool for addressing similar challenges in other contexts.

Despite its advantages, the proposed framework also faced certain limitations that warrant consideration. One notable limitation was the inability to achieve perfect accuracy. While the framework demonstrated high levels of accuracy, reaching 100% accuracy remained elusive. This limitation underscores the inherent complexity and variability of the problem domain, highlighting the need for continued refinement and optimization in future iterations of the framework. Another limitation pertains to the computational demands associated with ensemble learning. While ensemble learners offer significant benefits in accuracy and adaptability, they also incur computational overhead. Augmenting the number of classifiers while potentially enhancing accuracy can significantly increase memory and training time requirements. Balancing computational resources with accuracy becomes critical, especially when deploying the framework in real-world applications where efficiency is paramount. Additionally, the current framework does not explore the potential benefits of DL models. These models have shown promise in various domains for their ability to capture complex patterns and relationships in data. However, due to their computational intensity, they were not considered in this study, limiting the exploration of potentially superior solutions.

Future research efforts may benefit from integrating lightweight DL solutions into the ensemble learning framework to further enhance accuracy and performance and from evaluating the framework performance on diverse datasets to enhance generalizability.

CRediT authorship contribution statement

Zaid Allal: Methodology, Software, Formal analysis, Data curation, Writing – original draft. **Hassan N. Noura:** Conceptualization, Validation, Formal analysis, Resources, Data curation, Writing – review & editing, Visualization, Project administration. **Flavien Vernier:** Conceptualization, Validation, Formal analysis, Writing – review & editing, Supervision. **Ola Salman:** Methodology, Software, Formal analysis, Data curation, Writing – original draft. **Khaled Chahine:** Conceptualization, Validation, Formal analysis, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This paper was partially supported by funds from the project AAP Bourgogne-Franche-Comté Région 2022 dispositif ANER-IA-Limentaire, France and by the EIPHI Graduate School (contract "ANR-17-EURE-0002").

References

Advanced wind turbine drivetrain trends and opportunities department of energy. (2019). https://www.energy.gov/eere/articles/advanced-wind-turbine-drivetrain-trends-and-opportunities, (undefined 2/1/2024 11:58).

Alfian, Ganjar, Syafrudin, Muhammad, Fahrurrozi, Imam, Fitriyani, Norma Latif, Atmaji, Fransiskus Tatas Dwi, Widodo, Tri, et al. (2022). Predicting breast cancer from risk factors using SVM and extra-trees-based feature selection method. *Computers*, *11*(9), 136.

Allal, Zaid, Noura, Hassan N., & Chahine, Khaled (2024). Machine learning algorithms for solar irradiance prediction: A recent comparative study. *e-Prime - Advances in Electrical Engineering, Electronics and Energy*, *7*, Article 100453.

Allal, Zaid, Noura, Hassan N., Salman, Ola, & Chahine, Khaled (2024). Machine learning solutions for renewable energy systems: Applications, challenges, limitations, and future directions. *Journal of Environmental Management*, *354*, Article 120392.

Alshdaifat, Esra'a, Alshdaifat, Doa'a, Alsarhan, Ayoub, Hussein, Fairouz, & El-Salhi, Subhieh Moh'd Faraj S (2021). The effect of preprocessing techniques, applied to numeric features, on classification algorithms' performance. *Data*, *6*(2), 11.

Barella, Victor H., Garcia, Luís P. F., de Souto, Marcilio C. P., Lorena, Ana C., & de Carvalho, André C. P. L. F. (2021). Assessing the data complexity of imbalanced datasets. *Information Sciences*, *553*, 83–109.

Carbonneau, Real, Laframboise, Kevin, & Vahidov, Rustam (2008). Application of machine learning techniques for supply chain demand forecasting. *European Journal of Operational Research*, *184*(3), 1140–1154.

Carvalho, Thyago P, Soares, Fabrízzio AAMN, Vita, Roberto, Francisco, Roberto da P, Basto, João P, & Alcalá, Symone GS (2019). A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, *137*, Article 106024.

Chen, Tianqi, & Guestrin, Carlos (2024). XGBoost documentation. https://xgboost.readthedocs.io/en/stable/, Year Accessed, Accessed on March 27.

Chen, Tianqi, He, Tong, Benesty, Michael, Khotilovich, Vadim, Tang, Yuan, Cho, Hyunsu, et al. (2015). Xgboost: extreme gradient boosting. *1*, (4), (pp. 1–4). R package version 0.4-2.

Chen, Longting, Xu, Guanghua, Zhang, Qing, & Zhang, Xun (2019). Learning deep representation of imbalanced SCADA data for fault detection of wind turbines. *Measurement*, *139*, 370–379.

Cheng, Peng, Wang, Hai, Stojanovic, Vladimir, Liu, Fei, He, Shuping, & Shi, Kaibo (2022). Dissipativity-based finite-time asynchronous output feedback control for wind turbine system via a hidden Markov model. *International Journal of Systems Science*, *53*(15), 3177–3189.

Crone, Sven F., Lessmann, Stefan, & Stahlbock, Robert (2006). The impact of preprocessing on data mining: An evaluation of classifier sensitivity in direct marketing. *European Journal of Operational Research*, *173*(3), 781–800.

Cutler, Adele, Cutler, D. Richard, & Stevens, John R. (2012). Random forests. *Ensemble Machine Learning: Methods and Applications*, 157–175.

De Sousa, Pedro H Feijo, Navar de Medeiros, M, Almeida, Jefferson S, Reboucas Filho, Pedro P, de Albuquerque, Victor Hugo C, et al. (2019). Intelligent incipient fault detection in wind turbines based on industrial IoT environment. *Journal of Artificial Intelligence and Systems*, *1*(1), 1–19.

Dong, Xibin, Yu, Zhiwen, Cao, Wenming, Shi, Yifan, & Ma, Qianli (2020). A survey on ensemble learning. *Frontiers of Computer Science*, *14*, 241–258.

Dorogush, Anna, Gulin, Vladimir, & Matveyev, Gleb (2024). CatBoost documentation. https://catboost.ai/en/docs/, Accessed on March 27, 2024, Year Accessed.

Durocher, David B., & Feldmeier, Gerry R. (2004). Predictive versus preventive maintenance. *IEEE Industry Applications Magazine*, *10*(5), 12–21.

Forootan, Mohammad Mahdi, Larki, Iman, Zahedi, Rahim, & Ahmadi, Abolfazl (2022). Machine learning and deep learning in energy systems: A review. *Sustainability*, *14*(8), 4832.

Fu, Yichuan, Gao, Zhiwei, Liu, Yuanhong, Zhang, Aihua, & Yin, Xiuxia (2020). Actuator and sensor fault classification for wind turbine systems based on fast Fourier transform and uncorrelated multi-linear principal component analysis techniques. *Processes*, *8*(9), 1066.

Geurts, Pierre, Ernst, Damien, & Wehenkel, Louis (2006). Extremely randomized trees. *Machine Learning*, *63*, 3–42.

Gunning, David, Stefik, Mark, Choi, Jaesik, Miller, Timothy, Stumpf, Simone, & Yang, Guang-Zhong (2019). XAI—Explainable artificial intelligence. *Science Robotics*, *4*(37), eaay7120.

Helbing, Georg, & Ritter, Matthias (2018). Deep learning for fault detection in wind turbines. *Renewable and Sustainable Energy Reviews*, *98*, 189–198.

Hinton, Geoffrey E., & Roweis, Sam (2002). Stochastic neighbor embedding. *Advances in Neural Information Processing Systems*, *15*.

Hu, R Lily, Leahy, Kevin, Konstantakopoulos, Ioannis C, Auslander, David M, Spanos, Costas J, & Agogino, Alice M (2016). Using domain knowledge features for wind turbine diagnostics. In *2016 15th IEEE international conference on machine learning and applications* (pp. 300–307). IEEE.

Jabeur, Sami Ben, Gharib, Cheima, Mefteh-Wali, Salma, & Arfi, Wissal Ben (2021). CatBoost model and artificial intelligence techniques for corporate failure prediction. *Technological Forecasting and Social Change*, *166*, Article 120658.

Kumar, Atul, Khan, Muhammad, & Pandey, Bishwajeet (2018). Wind energy: A review paper. *Gyancity Journal of Engineering and Technology*, *4*, 29–37.

Kumar, Krishna, Kumar, Aman, Saini, Gaurav, Mohammed, Mazin Abed, Shah, Rachna, Nedoma, Jan, et al. (2024). Performance monitoring of kaplan turbine based hydropower plant under variable operating conditions using machine learning approach. *Sustainable Computing: Informatics and Systems*, Article 100958.

Leahy, Kevin, Hu, R Lily, Konstantakopoulos, Ioannis C, Spanos, Costas J, & Agogino, Alice M (2016). Diagnosing wind turbine faults using machine learning techniques applied to operational data. In *2016 ieee international conference on prognostics and health management (icphm)* (pp. 1–8). IEEE.

Leahy, Kevin, Hu, R Lily, Konstantakopoulos, Ioannis C, Spanos, Costas J, Agogino, Alice M, & O'Sullivan, Dominic TJ (2018). Diagnosing and predictingwind turbine faults from SCADA data using support vector machines. *International Journal of Prognostics and Health Management*, *9*(1).

Liu, Jiangyan, Zhang, Qing, Li, Xin, Li, Guannan, Liu, Zhongming, Xie, Yi, et al. (2021). Transfer learning-based strategies for fault diagnosis in building energy systems. *Energy and Buildings*, *250*, Article 111256.

Van der Maaten, Laurens, & Hinton, Geoffrey (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9*(11).

Miguelánez, Emilio, & Lane, David (2010). Predictive diagnosis for offshore wind turbines using holistic condition monitoring. In *OCEANS 2010 MTS/IEEE SEATTLe* (pp. 1–7). IEEE.

Moon, Seung-Hyun, Kim, Yong-Hyuk, Lee, Yong Hee, & Moon, Byung-Ro (2019). Application of machine learning to an early warning system for very short-term heavy rainfall. *Journal of Hydrology*, *568*, 1042–1054.

Nassif, Ali Bou, Talib, Manar Abu, Nasir, Qassim, & Dakalbab, Fatima Mohamad (2021). Machine learning for anomaly detection: A systematic review. *Ieee Access*, *9*, 78658–78700.

Nuwara, Yohanes (2023). IIoT wind turbine analytics. https://www.kaggle.com/code/yohanesnuwara/iiot-wind-turbine-analytics/input.

Ogunfowora, Oluwaseyi, & Najjaran, Homayoun (2023). Reinforcement and deep reinforcement learning-based solutions for machine maintenance planning, scheduling policies, and optimization. *Journal of Manufacturing Systems*, *70*, 244–263.

Paolanti, Marina, Romeo, Luca, Felicetti, Andrea, Mancini, Adriano, Frontoni, Emanuele, & Loncarski, Jelena (2018). Machine learning approach for predictive maintenance in industry 4.0. In *2018 14th IEEE/ASME international conference on mechatronic and embedded systems and applications* (pp. 1–6). IEEE.

Pedregosa, Fabian, Varoquaux, Gaël, Gramfort, Alexandre, Michel, Vincent, Thirion, Bertrand, Grisel, Olivier, et al. (2011). Scikit-learn: Machine learning in Python. *The Journal of machine Learning research*, *12*, 2825–2830.

Ran, Yongyi, Zhou, Xin, Lin, Pengfeng, Wen, Yonggang, & Deng, Ruilong (2019). A survey of predictive maintenance: Systems, purposes and approaches. arXiv preprint arXiv:1912.07383.

Sagi, Omer, & Rokach, Lior (2018). Ensemble learning: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *8*(4), Article e1249.

Saidur, Rahman, Rahim, Nasrudin A, Islam, Monirul Rafiqu, & Solangi, Khalid H (2011). Environmental impact of wind energy. *Renewable and Sustainable Energy Reviews*, *15*(5), 2423–2430.

Segger, Marie-Claire Cordonier (2016). Advancing the Paris agreement on climate change for sustainable development. *Cambridge International Law Journal*, *5*(2), 202–237.

Song, Xiaona, Sun, Peng, Song, Shuai, & Stojanovic, Vladimir (2023). Finite-time adaptive neural resilient DSC for fractional-order nonlinear large-scale systems against sensor-actuator faults. *Nonlinear Dynamics*, 1–16.

Stojanović, Vladimir (2023). Fault-tolerant control of a hydraulic servo actuator via adaptive dynamic programming. *Mathematical Modelling and Control*, *3*, 181–191.

Sun, Tongda, Yu, Gang, Gao, Mang, Zhao, Lulu, Bai, Chen, & Yang, Wanqian (2021). Fault diagnosis methods based on machine learning and its applications for wind turbines: A review. *IEEE Access*, *9*, 147481–147511.

Susto, Gian Antonio, Schirru, Andrea, Pampuri, Simone, McLoone, Seán, & Beghi, Alessandro (2014). Machine learning for predictive maintenance: A multiple classifier approach. *IEEE Transactions on Industrial Informatics*, *11*(3), 812–820.

Taud, Hind, & Mas, J. F. (2018). Multilayer perceptron (MLP). *Geomatic Approaches for Modeling Land Change Scenarios*, 451–455.

Tawfiq, Kotb B., Mansour, Arafa S., Ramadan, Haitham S., Becherif, Mohamed, & Elkholy, E. E. (2019). Wind energy conversion system topologies and converters: Comparative review. *Energy Procedia*, *162*, 38–47, Emerging and Renewable Energy: Generation and Automation.

US Department of Energy (2023). How do wind turbines work?.

Uysal, Alper Kursat, & Gunal, Serkan (2014). The impact of preprocessing on text classification. *Information Processing & Management*, *50*(1), 104–112.

Wang, Peng, Poovendran, Parthasarathy, & Manokaran, Karthik Bala (2021). Fault detection and control in integrated energy system using machine learning. *Sustainable Energy Technologies and Assessments*, *47*, Article 101366.

Wang, Rui, Zhuang, Zhihe, Tao, Hongfeng, Paszke, Wojciech, & Stojanovic, Vladimir (2023). Q-learning based fault estimation and fault tolerant iterative learning control for MIMO systems. *ISA Transactions*, *142*, 123–135.

Wind Energy Technologies Office (0000). Frequently Asked Questions about Wind Energy | Department of Energy, https://www.energy.gov/eere/wind/frequently-asked-questions-about-wind-energy.

Xia, Liqiao, Zheng, Pai, Li, Jinjie, Huang, Xiao, & Gao, Robert X (2023). Histogram-based gradient boosting tree: A federated learning approach for collaborative fault diagnosis. *IEEE/ASME Transactions on Mechatronics*.

Xiang, Ling, Wang, Penghe, Yang, Xin, Hu, Aijun, & Su, Hao (2021). Fault detection of wind turbine based on SCADA data analysis using CNN and LSTM with attention mechanism. *Measurement*, *175*, Article 109094.

Zhang, Dongmei, Yuan, Jun, Zhu, Jiang, Ji, Qingchang, Zhang, Xintong, Liu, Hao, et al. (2020). Fault diagnosis strategy for wind turbine generator based on the Gaussian process metamodel. *Mathematical Problems in Engineering*, *2020*.